

AI Harness / AI PDE / AI Infra

企业级完整学习方案、视频教程与落地操作手册

36 周逐周学习路线

GPU 物理集群、国产 GPU、千卡规划、公有云与模型部署调优

资料检索与链接核验日期：2026-10-04

适用于企业 PoC、内部试点、平台建设与岗位转型

版本：1.1

目录

使用说明	7
推荐使用方式	7
第一部分：36 周视频教程与逐周操作手册	8
36 周完整视频学习方案与逐周操作手册	8
0. 怎么使用这份手册	8
第一阶段：工程与 AI 基础（第 1~4 周）	8
第 1 周：Python、Git、HTTP/JSON/SSE	8
第 2 周：LLM 基础、Token、Prompt、Context	9
第 3 周：FastAPI、Pydantic、Docker	9
第 4 周：Postgres、Redis、状态恢复、基础观测	10
第二阶段：AI Harness（第 5~12 周）	10
第 5 周：Harness、Agent Loop、预算与停止条件	10
第 6 周：Context Engineering 与压缩	11
第 7 周：Memory、RAG、Embedding、向量数据库	11
第 8 周：Function Calling、MCP、工具调用	12
第 9 周：工具设计、幂等、权限与审计	12
第 10 周：Sandbox、Human-in-the-Loop、审批	12
第 11 周：Verification、Critic、Agent 评测	13
第 12 周：长任务、跨 Session 恢复与多 Agent	13
第三阶段：AI PDE 产品工程（第 13~16 周）	14
第 13 周：用户研究、JTBD、产品指标	14
第 14 周：AI PRD、状态机、API 契约	14
第 15 周：前端、流式交互、可访问性	15
第 16 周：评测、灰度、上线、复盘	15
第四阶段：AI Infra 单机与 Kubernetes（第 17~20 周）	16
第 17 周：GPU、CUDA、vLLM 单机部署	16
第 18 周：KV Cache、PagedAttention、Continuous Batching	16
第 19 周：量化、Prefix Cache、推测解码	17
第 20 周：Kubernetes、GPU Operator、模型网关、自动扩缩容	17
第五阶段：分布式推理与调度（第 21~24 周）	18
第 21 周：Kueue、Ray、KServe	18

第 22 周: TP、PP、DP、NCCL	18
第 23 周: Dynamo、llm-d、Prefill / Decode 分离	19
第 24 周: 压测、容量、SLO 与成本	19
第六阶段: 观测、评估、安全与可靠性 (第 25 ~ 28 周)	19
第 25 周: OpenTelemetry、Langfuse、MLflow、监控	19
第 26 周: RAG、Agent、安全评测	20
第 27 周: OWASP、NIST、安全与合规	20
第 28 周: 可靠性、混沌、成本和事故复盘	21
第七阶段: 毕业项目 (第 29 ~ 32 周)	21
第 29 周: 选题、范围、评测基线	21
第 30 周: 端到端开发	22
第 31 周: 压测、调优、安全、故障演练	22
第 32 周: 答辩、README、演示和复盘	22
第八阶段: GPU 物理集群专项 (第 33 ~ 36 周)	23
第 33 周: GPU 服务器选型、机柜、供电、散热	23
第 34 周: BMC、网络、存储、裸机初始化	23
第 35 周: 驱动、CUDA、NCCL、K8s / Slurm、DCGM	24
第 36 周: Burn-in、故障演练、验收和调优	24
附录 A: 视频失效后的检索公式	25
附录 B: 每门视频的学习记录模板	25
附录 C: 学习完成的最低标准	26
第二部分: AI Harness 企业落地教程	27
AI Harness 企业落地教程	27
1. Harness 到底是什么	27
2. 参考架构	28
3. 建议项目结构	29
4. 八个递进实验	29
5. AGENTS.md 与 Skills 最小模板	35
6. Harness 安全基线	35
7. 完成标准	35
第三部分: AI PDE 产品工程落地教程	37
AI PDE 产品工程落地教程	37

1. PDE 在企业里到底负责什么	37
2. PDE 工作流	37
3. 七个递进实验	39
4. AI PRD 模板	43
5. PDE 与其他角色的协作边界	44
6. 完成标准	44
第四部分：AI Infra 企业平台教程	45
AI Infra 企业平台落地教程	45
1. AI Infra 的边界	45
2. 参考架构	45
3. 十三个递进实验	46
4. 企业 SLO 模板	52
5. 完成标准	52
第五部分：GPU 物理集群建设、选型、部署与调优	53
GPU 物理集群建设、选型、部署与调优	53
1. GPU 物理集群的完整分层	53
2. 先写需求，再做硬件选型	53
3. GPU 与服务器选型	54
4. 机柜、供电与散热	54
5. 网络设计	55
6. 存储设计	56
7. 软件栈与节点初始化	57
8. Slurm 还是 Kubernetes	57
9. 模型部署路径	58
10. 调优路径：从物理层到应用层	58
11. 4 周物理集群专项（可接在 32 周路线后）	59
12. 物理集群交付清单	60
第六部分：GPU 集群监控、调试、故障处理与验收	61
GPU 集群监控、调试、故障处理与验收	61
1. 运维闭环	61
2. 监控体系	61
3. 基线检查命令	62
4. 验收与 Burn-in 测试	63
5. 故障分类与处理流程	65

6. 常见故障 Runbook	66
7. 节点 Drain、维修和回池	69
8. 监控看板建议	69
9. 验收报告模板	70
10. 最终检查表	70
第七部分：企业落地常见坑与注意事项	71
企业落地常见坑与注意事项	71
1. 通用原则	71
2. 分模块避坑表	71
3. 企业上线红线	76
4. P0 上线检查清单	76
5. 配套视频与资料	76
第八部分：国产 GPU 与千卡集群规划实施指南	77
国产 GPU 与千卡集群规划实施指南	77
1. 国产 GPU 集群与通用 GPU 集群的差异	77
2. 主流国产算力生态地图	77
3. 国产 GPU 选型方法	78
4. 千卡集群规划	79
5. 分阶段实施路线	81
6. 国产软件栈迁移要点	81
7. 千卡集群验收	82
8. 视频与资料	82
9. 最终检查表	83
第九部分：公有云 GPU 选型与实施要点	84
公有云 GPU 选型与实施要点	84
1. 公有云 GPU 与自建 GPU 集群	84
2. 选型维度	84
3. 主要公有云 GPU 入口	85
4. 公有云千卡集群规划	86
5. 成本控制	87
6. 公有云安全	87
7. 实施步骤	87
8. 常见坑	88
9. 视频与资料	88

10. 最终检查表	88
第十部分：32 周完整学习路线（含物理集群扩展说明）	89
AI Harness / PDE / Infra 32 周完整学习路线	89
1. 能力目标	89
2. 每周固定节奏	89
3. 前置要求	89
4. 32 周路线	90
5. 每周实验记录模板	91
6. 阶段通关规则	92
7. 最终答辩的 12 个问题	92
第十一部分：毕业项目 SOP 与验收	93
毕业项目 SOP 与验收	93
1. 项目目标	93
2. 毕业项目必须覆盖	93
3. 推荐目录	93
4. 端到端验收流程	94
5. 必须提交的文件	96
6. 评分表	96
7. 答辩四分钟结构	96
第十二部分：最新资讯与来源索引	97
最新资讯与来源索引	97
一、AI Harness / Agent Runtime	97
二、AI PDE：产品设计 / 产品开发工程	98
三、AI Infra：推理、编排、调度和平台	98
四、GPU 物理集群：选型、建设、调优、监控和故障	100
五、国产 GPU、千卡集群与公有云	101
六、资料可信度规则	101
七、高频版本风险	102

使用说明

本 PDF 将 36 周视频学习路线、AI Harness、AI PDE、AI Infra、GPU 物理集群和毕业项目合并为一份完整手册。视频用于建立直觉，官方文档用于确认接口，实验用于形成可复现证据。每个学习模块都按照“主视频、备选视频、官方文档、操作步骤、验收产物”组织。

注意：视频平台内容可能变化；本 PDF 完成时的链接核验日期为 2026-10-04。部署前必须重新核对官方 release notes、兼容矩阵和企业安全基线。

推荐使用方式

- 每周学习 10 ~ 12 小时，固定完成理论、编码、压测和复盘。
- 每个实验保留代码、命令、截图、指标、失败案例和结论。
- 先完成最小闭环，再替换为 Ray、KServe、llm-d、Dynamo 和物理集群。
- 所有模型、工具、Prompt、Skill 和集群版本必须锁定并记录。
- 毕业项目必须同时覆盖产品指标、Harness 控制面、推理平台和 GPU 基础设施。

第一部分：36 周视频教程与逐周操作手册

来源文件：09_36周视频教程与逐周操作手册.md

36 周完整视频学习方案与逐周操作手册

检索日期：**2026-10-04**。视频平台的内容和链接会变化，学习时以“视频建立直觉，官方文档确认接口，实验验证结论”为原则。

这里的“每个知识点都有视频”按下面的标准执行：**每个学习模块至少有 1 个主视频、1 个备选视频、1 份官方文档、1 个可执行实验、1 个可验收产物**。一条视频可以覆盖多个相邻知识点。

0. 怎么使用这份手册

每周只做四件事：

1. 看主视频，写出 5 条自己的话，不抄字幕；
2. 看官方文档，核对命令、参数和版本；
3. 按“操作步骤”做实验；
4. 用“验收产物”证明自己真的完成。

统一学习仓库：

```
ai-learning-lab/  
├── foundation/  
├── harness/  
├── pde/  
├── infra/  
├── gpu-cluster/  
├── evals/  
├── benchmarks/  
├── reports/  
└── README.md
```

第一阶段：工程与 AI 基础（第 1~4 周）

第 1 周：Python、Git、HTTP/JSON/SSE

视频

- 主视频：[3 小时超快速入门 Python](#)
- 补充：[Git + GitHub 核心概念全攻略](#)
- 补充：[1 小时全面入门 HTTP 协议](#)
- 官方：[Python Docs](#)、[Git Book](#)、[MDN HTTP](#)

操作步骤

1. 安装 Python 3.12、Git、VS Code 或 Cursor。
2. 创建 ai-learning-lab，配置 Git 用户和远程仓库。
3. 用 Python 写一个 CLI：接收 URL、超时、重试次数。
4. 用 requests 或 httpx 调用一个公开 API，处理 JSON。
5. 用 http.server 起一个本地接口，理解 Request / Response / Header。
6. 用 SSE 模拟服务端逐字输出。

7. 做一次 Git 分支、Commit、Push、PR、回滚。

验收产物

- CLI 代码;
- HTTP 请求日志;
- SSE Demo;
- `foundation/week01.md`: 状态码、JSON、SSE 的区别。

第 2 周：LLM 基础、Token、Prompt、Context

视频

- 主视频: [一次性搞懂 Agent 到底怎么用](#)
- 补充: [LLM Context Engineering Bootcamp Lecture 1](#)
- 官方: [OpenAI Prompting Guide](#)
- 官方: [OpenAI Text Generation](#)

操作步骤

1. 调用模型 API, 观察 System / User / Assistant 消息结构。
2. 把同一问题分别用零样本、Few-shot、角色提示、结构化输出提示调用。
3. 记录每次请求的输入 Token、输出 Token、延迟、费用。
4. 比较 Temperature 0、0.2、0.7、1.0 的结果。
5. 构造 5 组测试问题, 做 Prompt 版本表。
6. 实验上下文长度增加时, TTFT 和成本如何变化。

验收产物

- Prompt 版本表;
- Token / 延迟 / 成本 CSV;
- 一份“Prompt 不是玄学”的实验报告。

第 3 周：FastAPI、Pydantic、Docker

视频

- 主视频: [FastAPI 从入门到实战](#)
- 补充: [Pydantic 数据结构与验证](#)
- 补充: [40 分钟 Docker 实战攻略](#)
- 官方: [FastAPI](#)、[Pydantic](#)、[Docker](#)

操作步骤

1. 用 `uv init` 创建项目, 建立虚拟环境。
2. 编写 `/health`、`/chat`、`/runs/{id}` 三个接口。
3. 用 Pydantic 定义请求、响应、错误 Schema。
4. 加入超时、重试、异常处理和请求 ID。
5. 写 Dockerfile, 使用非 root 用户。
6. 用 Docker Compose 启动 API 和 Redis。

验收产物

- OpenAPI 页面截图;
- Dockerfile;
- Compose 文件;
- API 单元测试。

第 4 周：Postgres、Redis、状态恢复、基础观测

视频

- 主视频: [Prometheus + Grafana 可观测性实战](#)
- 补充: [10 分钟了解 MLOps 与 CI/CD](#)
- 官方: [PostgreSQL Docs](#)、[Redis Docs](#)、[OpenTelemetry](#)

操作步骤

1. 用 Postgres 保存 Run、Step、Artifact、Feedback。
2. 用 Redis 保存短期会话和锁。
3. 模拟进程重启, 验证 Run 能从数据库恢复。
4. 加入结构化日志, 包含 `run_id`、`tenant_id`、`step`。
5. 用 Prometheus 指标暴露请求数和延迟。
6. 配置一个本地 Grafana 面板。

验收产物

- 数据表设计;
- 状态恢复测试;
- 日志样例;
- 本地监控面板。

第二阶段：AI Harness（第 5 ~ 12 周）

第 5 周：Harness、Agent Loop、预算与停止条件

视频

- 主视频: [Let's build a Coding Agent Harness from Scratch](#)
- 补充: [Building Effective Agents](#)
- 官方: [Microsoft Agent Harness](#)
- 官方: [OpenAI Agents API](#)

操作步骤

1. 定义 `AgentSpec`、`RunState`、`ToolCall`。
2. 实现 `Model` → `Tool` → `Model` 循环。
3. 加最大步骤、最大时间、最大 Token、最大成本。
4. 加工具超时和异常处理。
5. 保存每一步的输入、输出和耗时。
6. 用真实模型跑 10 个任务, 记录失败类型。

验收产物

- `controller.py`;
- 10 个成功 / 失败任务;
- `reports/week05.md`。

第 6 周：Context Engineering 与压缩

视频

- 主视频: [LLM Context Engineering Series Part 1](#)
- 补充: [Context Engineering Lecture 2: System Prompts](#)
- 官方: [Anthropic Context Engineering](#)

操作步骤

1. 把上下文拆成 System、Goal、Recent、Summary、Retrieved、Tool Results。
2. 实现最近 N 轮保留和旧消息摘要。
3. 对长工具结果做截断、结构化提炼和引用保留。
4. 对敏感字段做脱敏。
5. 对比不压缩、摘要、RAG 检索三种上下文策略。
6. 记录 Token、延迟、任务成功率。

验收产物

- Context Builder;
- 压缩前后指标;
- 上下文结构图。

第 7 周：Memory、RAG、Embedding、向量数据库

视频

- 主视频: [RAG 新手完整入门](#)
- 补充: [Embedding 与向量数据库](#)
- 官方: [pgvector](#)、[Milvus Docs](#)

操作步骤

1. 准备 20 ~ 50 份 Markdown / PDF 文档。
2. 实现文档解析、分块、Embedding、入库。
3. 实现 Top-K 向量检索。
4. 加入 BM25 关键词检索，做混合检索。
5. 加入 Rerank。
6. 做一个带引用的 RAG API。
7. 建立 20 条 Golden Questions。

验收产物

- RAG Demo;
- 检索评测表;
- 引用展示页面或 CLI。

第 8 周：Function Calling、MCP、工具调用

视频

- 主视频: [Function Calling 技术详解](#)
- 补充: [MCP 终极指南：从原理到实战](#)
- 补充: [MCP 保姆级原理与实战](#)
- 官方: [Model Context Protocol](#)

操作步骤

1. 用 JSON Schema 定义一个只读天气 / 知识库工具。
2. 让模型产生结构化 Tool Call。
3. 实现 Tool Executor，加入超时、重试、错误码。
4. 创建最小 MCP Server。
5. 通过 MCP Client 调用工具。
6. 把工具名称、版本、风险级别写入 Registry。

验收产物

- Tool Registry;
- MCP Server;
- 调用日志和失败案例。

第 9 周：工具设计、幂等、权限与审计

视频

- 主视频: [Writing effective tools for agents](#)
- 补充: [MCP 从原理到实战](#)
- 官方: [OpenAI Function Calling](#)

操作步骤

1. 给工具加 `risk`、`version`、`required_scopes`。
2. 写操作增加幂等键。
3. 失败返回稳定错误码，不返回堆栈。
4. 对工具结果做大小限制和 Token 优化。
5. 记录参数摘要、调用者、耗时、结果状态。
6. 做越权调用测试。

验收产物

- 工具设计规范;
- 幂等测试;
- 权限测试报告。

第 10 周：Sandbox、Human-in-the-Loop、审批

视频

- 主视频: [3 reasons sandboxing won't secure your AI agent](#)
- 补充: [Agent Harness 相关安全实践](#)

- 官方: [OWASP LLM Top 10](#)

操作步骤

1. 用 Docker 隔离代码执行。
2. 限制文件系统、网络、CPU、内存和时间。
3. 实现审批状态机:

RUNNING → WAITING_APPROVAL → APPROVED / REJECTED / TIMEOUT。

4. 对写数据库、发邮件、修改工单强制审批。
5. 记录审批人、理由、参数快照。
6. 做 Prompt Injection 越权测试。

验收产物

- 沙箱配置;
- 审批服务;
- 安全测试记录。

第 11 周: Verification、Critic、Agent 评测

视频

- 主视频: [The agent evaluation revolution](#)
- 补充: [LLM as a Judge: Scaling AI Evaluation](#)
- 官方: [OpenAI Evals](#)、[promptfoo](#)

操作步骤

1. 为每个任务定义机器可判定的通过条件。
2. 实现 Schema 校验、单元测试、引用检查。
3. 加入 Critic / 二次生成 / 修复循环。
4. 构建 30 条正常用例、20 条边界用例、20 条对抗用例。
5. 每次变更自动跑回归。
6. 输出新旧版本对比。

验收产物

- 评测数据集;
- 回归门禁;
- 评测报告。

第 12 周: 长任务、跨 Session 恢复与多 Agent

视频

- 主视频: [Effective harnesses for long-running agents](#)
- 补充: [Agent Memory Explained](#)
- 官方: [Anthropic Long-running Agents](#)

操作步骤

1. 实现 Initializer Agent 和 Worker Agent。
2. 生成 progress.md、任务清单、验收标准。

3. 每次运行结束写回状态和产物位置。
4. 强制 5 次上下文重置，验证跨 Session 恢复。
5. 引入一个受控 Subagent，做任务分发和结果聚合。
6. 记录重复工作、停滞和死循环。

验收产物

- 长任务 Harness;
- 5 次恢复日志;
- 多 Agent 架构图。

第三阶段：AI PDE 产品工程（第 13 ~ 16 周）

第 13 周：用户研究、JTBD、产品指标

视频

- 主视频: [The ultimate guide to JTBD](#)
- 补充: [One-Hour Product Analytics Tutorial](#)
- 官方: [Google PAIR Guidebook](#)

操作步骤

1. 选一个真实业务流程。
2. 访谈 3 位目标用户，记录当前步骤、耗时和错误。
3. 写 JTBD、用户故事、主指标、护栏指标。
4. 基线化：耗时、错误率、人工成本。
5. 明确 AI 不做的部分和人工接管条件。

验收产物

- `discovery.md`;
- 用户旅程;
- 指标基线。

第 14 周：AI PRD、状态机、API 契约

视频

- 主视频: [How to Write a Product Requirements Document](#)
- 补充: [How to write a great PRD](#)
- 官方: [Nielsen Norman Group AI Chat UX](#)

操作步骤

1. 写 Problem、User、Goal、Non-goal。
2. 画 AI 状态机：理解、检索、规划、工具、审批、失败、完成。
3. 定义 API 请求、响应、流式事件和错误码。
4. 定义引用、拒答、低置信度和人工接管策略。
5. 写 SLO 和成本上限。

验收产物

- AI PRD;
- 状态机;
- OpenAPI 契约。

第 15 周：前端、流式交互、可访问性

视频

- 主视频: [Learn React JS Full Tutorial](#)
- 补充: [Learn Accessibility Full a11y Tutorial](#)
- 官方: [React](#)、[Next.js](#)

操作步骤

1. 用 Next.js 创建页面。
2. 实现流式输出、停止生成、重试。
3. 设计等待、检索、工具执行、审批、失败状态。
4. 显示引用、置信度、成本和下一步动作。
5. 加入键盘操作、屏幕阅读器标签和错误提示。
6. 用真实延迟测试体验。

验收产物

- 前端原型;
- 状态截图;
- 无障碍检查报告。

第 16 周：评测、灰度、上线、复盘

视频

- 主视频: [Product Analytics Tutorial](#)
- 补充: [How To A/B Test a Product](#)
- 官方: [OpenTelemetry](#)、[Langfuse](#)

操作步骤

1. 建立 Golden Set、边界集、对抗集。
2. 接 Harness API，前后端贯通。
3. 用 Feature Flag 做 1% 灰度。
4. 监控任务成功、接管率、延迟、成本和投诉。
5. 做一次 A/B 或前后对比。
6. 完成上线复盘。

验收产物

- 灰度方案;
- 上线看板;
- 复盘报告。

第四阶段：AI Infra 单机与 Kubernetes（第 17 ~ 20 周）

第 17 周：GPU、CUDA、vLLM 单机部署

视频

- 主视频：[CUDA 编程基础入门系列](#)
- 补充：[Understanding vLLM with a Hands-On Demo](#)
- 补充：[vLLM 快速入门](#)
- 官方：[vLLM Docs](#)、[CUDA Toolkit Docs](#)

操作步骤

1. 检查驱动、CUDA、显存和拓扑：`nvidia-smi`、`nvidia-smi topo -m`。
2. 安装 vLLM，启动 Qwen2.5-7B-Instruct。
3. 用 `/v1/chat/completions` 测试流式和非流式。
4. 调整 `--gpu-memory-utilization`、`--max-model-len`。
5. 观察 OOM、显存预分配和并发变化。
6. 记录 TTFT、Token/s、显存。

验收产物

- vLLM 服务；
- 参数实验表；
- `infra/week17.md`。

第 18 周：KV Cache、PagedAttention、Continuous Batching

视频

- 主视频：[PagedAttention：KV Cache 与 Agent 工程](#)
- 补充：[保姆级 KV Cache 教程](#)
- 补充：[PagedAttention: Behind vLLM's Insane Speed](#)
- 官方：[vLLM Paged Attention Design](#)

操作步骤

1. 推导 KV Cache 显存公式。
2. 对比不同 `max-model-len` 下的显存和并发。
3. 做并发 1、4、8、16 的连续批处理实验。
4. 记录吞吐、单请求延迟、队列时间。
5. 解释 PagedAttention 和 Block Table。
6. 画出 Prefill / Decode 流程图。

验收产物

- KV Cache 计算表；
- 压测报告；
- 原理解释图。

第 19 周：量化、Prefix Cache、推测解码

视频

- 主视频: [LLM Quantization Explained](#)
- 补充: [vLLM Prefix Caching Tutorial](#)
- 补充: [Faster LLMs: Speculative Decoding](#)
- 官方: [vLLM Optimization](#)

操作步骤

1. 对比 BF16、AWQ、GPTQ 的显存、吞吐、质量。
2. 用重复 system prompt 对比 Prefix Cache 开 / 关。
3. 测试推测解码的吞吐和延迟。
4. 记录每百万 Token 成本。
5. 确定默认生产配置和回退配置。

验收产物

- 量化对比报告；
- Prefix Cache 报告；
- 生产配置建议。

第 20 周：Kubernetes、GPU Operator、模型网关、自动扩缩容

视频

- 主视频: [NVIDIA GPU Operator 架构与部署](#)
- 补充: [vLLM on Kubernetes in Production](#)
- 官方: [GPU Operator](#)

操作步骤

1. 安装 Kubernetes、GPU Operator、Network Operator。
2. 确认节点 GPU 资源和 Device Plugin。
3. 部署 vLLM Deployment / Service。
4. 接入 LiteLLM 或其他模型网关。
5. 配置多副本、健康检查、滚动升级和回滚。
6. 测试 HPA / KEDA 或 KServe 自动扩缩容。

验收产物

- K8s YAML / Helm；
- 网关配置；
- 扩容 / 回滚记录。

第五阶段：分布式推理与调度（第 21 ~ 24 周）

第 21 周：Kueue、Ray、KServe

视频

- 主视频: [Intro to Kueue](#)
- 补充: [Fast LLM Inference by vLLM and KServe](#)
- 官方: [Ray Serve LLM](#)、[Kueue](#)、[KServe](#)

操作步骤

1. 配置两个团队队列和 GPU 配额。
2. 用 Ray Serve 部署一个模型服务。
3. 用 KServe 部署同一模型，做 Canary。
4. 验证优先级、排队、抢占。
5. 比较 Ray Serve 和 KServe 的运维边界。

验收产物

- 队列 / 配额实验；
- 两套部署方案；
- 选型矩阵。

第 22 周：TP、PP、DP、NCCL

视频

- 主视频: [分布式训练：张量并行、流水线并行原理](#)
- 补充: [千张显卡的高效协同：TP、PP 与 3D 并行](#)
- 补充: [NCCL 通信库入门](#)
- 官方: [vLLM Parallelism and Scaling](#)

操作步骤

1. 做单卡基线。
2. 启动 TP=2，记录加速比。
3. 启动 PP=2，记录流水线气泡。
4. 运行 `nccl-tests`。
5. 对比 NVLink、PCIe、跨机网络。
6. 解释为什么两张卡不等于两倍性能。

验收产物

- 多卡基准；
- NCCL 数据；
- 扩展效率公式和结论。

第 23 周：Dynamo、llm-d、Prefill / Decode 分离

视频

- 主视频: [Distributed Inference 101: Getting Started with NVIDIA Dynamo](#)
- 补充: [LLM-D Explained](#)
- 补充: [Why splitting prefill and decode doubles throughput](#)
- 官方: [NVIDIA Dynamo、llm-d](#)

操作步骤

1. 部署一个 llm-d Well-Lit Path 或 Dynamo 示例。
2. 打开 KV-aware routing。
3. 对比普通 Round Robin 和缓存感知路由。
4. 做 Prefix Cache 命中实验。
5. 记录 TTFT、吞吐、GPU 利用率、成本。

验收产物

- 分布式推理部署；
- 路由对比；
- 性能报告。

第 24 周：压测、容量、SLO 与成本

视频

- 主视频: [vLLM 并发压测与监控实战](#)
- 补充: [GPU Monitoring with DCGM, Prometheus and Grafana](#)
- 官方: [vLLM Benchmarking](#)

操作步骤

1. 用并发 1、4、8、16、32、64 压测。
2. 记录 TTFT、TPOT、QPS、错误、显存、KV Cache。
3. 计算每百万 Token 成本。
4. 算出容量上限和安全水位。
5. 写 SLO 和降级策略。

验收产物

- Benchmark 报告；
- SLO；
- 成本表。

第六阶段：观测、评估、安全与可靠性（第 25 ~ 28 周）

第 25 周：OpenTelemetry、Langfuse、MLflow、监控

视频

- 主视频: [LLM Observability with OpenTelemetry](#)

- 补充: [MLflow Model Tracking and Registry](#)
- 官方: [OTel GenAI](#)、[Langfuse](#)、[MLflow GenAI](#)

操作步骤

1. 为一次 Run 生成 `trace_id`。
2. 记录模型、Prompt 版本、Token、工具、检索、审批。
3. 接入 Langfuse 或 MLflow。
4. 建立延迟、成本、错误看板。
5. 从一条失败 Trace 追到具体工具和版本。

验收产物

- Trace 截图;
- 统一 Dashboard;
- 错误归因记录。

第 26 周: RAG、Agent、安全评测

视频

- 主视频: [Key Metrics and Evaluation Methods for RAG](#)
- 补充: [The agent evaluation revolution](#)
- 官方: [Ragas](#)、[promptfoo](#)、[DeepEval](#)

操作步骤

1. 建立 RAG 检索指标和生成指标。
2. 建立 Agent 任务成功率、工具成功率、接管率。
3. 建立 Prompt Injection、越权、数据泄露测试。
4. 配置 CI 回归门禁。
5. 输出版本对比报告。

验收产物

- 评测集;
- 回归流水线;
- 评测报告。

第 27 周: OWASP、NIST、安全与合规

视频

- 主视频: [OWASP Top 10 for LLM Applications](#)
- 补充: [NIST AI RMF](#)
- 官方: [OWASP LLM Top 10](#)、[NIST AI RMF](#)

操作步骤

1. 做资产、数据流和信任边界图。
2. 对 Prompt Injection、敏感信息、过度代理、供应链建威胁模型。
3. 配置 RBAC、Secrets、工具 allowlist、DLP、审计。
4. 做红队测试。

5. 记录残留风险和修复计划。

验收产物

- Threat Model;
- 红队报告;
- 控制清单。

第 28 周：可靠性、混沌、成本和事故复盘

视频

- 主视频: [GPU Monitoring That Actually Helps](#)
- 补充: [LLM Observability](#)
- 官方: [Prometheus](#)、[Grafana](#)

操作步骤

1. 杀死一个模型副本，观察恢复。
2. 注入工具超时、数据库抖动和队列积压。
3. 验证熔断、降级、重试和人工接管。
4. 做成本异常演练。
5. 写事故复盘模板和 Runbook。

验收产物

- 故障演练记录;
- Runbook;
- 成本告警。

第七阶段：毕业项目（第 29 ~ 32 周）

第 29 周：选题、范围、评测基线

视频

- 主视频: [AI Product Management Complete Course](#)
- 补充: [How to Write a PRD](#)

操作步骤

1. 从企业知识助手、运维 Copilot、数据分析 Agent 中选一个。
2. 写用户、问题、范围、非目标和成功指标。
3. 建立 baseline。
4. 选模型、Harness、工具、数据、部署方式。
5. 画端到端架构图。

验收产物

- 项目 Spec;
- Baseline;
- 架构图。

第 30 周：端到端开发

视频

- 主视频: [OpenAI Agents SDK Full Series](#)
- 补充: [LangGraph Complete Course](#)

操作步骤

1. 完成 API、Harness、工具、记忆、前端。
2. 接入 Trace 和评测。
3. 完成审批和失败恢复。
4. 完成模型网关和 vLLM。
5. 跑通正常、失败、对抗三条路径。

验收产物

- 可运行系统；
- Commit 历史；
- 单元 / 集成测试。

第 31 周：压测、调优、安全、故障演练

视频

- 主视频: [Optimize LLM inference with vLLM](#)
- 补充: [GPU Monitoring with DCGM](#)

操作步骤

1. 做并发和长上下文压测。
2. 优化 TP、Prefix Cache、Batch、量化。
3. 做安全、权限和数据泄露测试。
4. 做副本故障和降级演练。
5. 记录吞吐、延迟、成本和恢复时间。

验收产物

- Benchmark；
- 安全报告；
- 故障报告。

第 32 周：答辩、README、演示和复盘

视频

- 主视频: [AI Product Engineer Role](#)
- 补充: [AI Product Management Course](#)

操作步骤

1. 写 README、架构图、启动命令。
2. 录一个 4 分钟演示。
3. 准备 12 个答辩问题。

4. 展示正常、审批、失败恢复、故障切换。
5. 写改进计划。

验收产物

- 毕业项目仓库；
- 演示视频；
- 评分表。

第八阶段：GPU 物理集群专项（第 33 ~ 36 周）

第 33 周：GPU 服务器选型、机柜、供电、散热

视频

- 主视频： [NVIDIA Certified Associate AI Infrastructure and Operations](#)
- 补充： [NVIDIA DGX SuperPOD 架构](#)
- 补充： [Liquid Cooling in AI Data Center](#)
- 官方： [NVIDIA DGX SuperPOD Docs](#)

操作步骤

1. 按 workload 估算 GPU 显存、带宽、互联和数量。
2. 选择 GPU、CPU、内存、NIC、本地盘。
3. 计算机柜功率、UPS、PDU、制冷和承重。
4. 确定风冷或液冷方案。
5. 画机柜图、端口表、IP 规划。
6. 写 BOM 和验收标准。

验收产物

- 选型报告；
- BOM；
- 机柜图 / 端口表；
- 功率与散热计算。

第 34 周：BMC、网络、存储、裸机初始化

视频

- 主视频： [Redfish School](#)
- 补充： [OpenBMC Overview](#)
- 补充： [Spine and Leaf Architecture](#)
- 补充： [InfiniBand 与 RoCE： AI Data Center](#)
- 官方： [OpenBMC](#)、[MAAS](#)、[Tinkerbell](#)

操作步骤

1. 隔离 BMC 带外网络，修改默认密码。
2. 用 Redfish 完成 BIOS、RAID、Boot 配置。
3. 用 MAAS / Tinkerbell / PXE 安装 OS。

4. 配置管理网、计算网、存储网、带外网。
5. 配置 RDMA、GID、MTU、PFC / ECN。
6. 挂载并行文件系统或对象存储。
7. 完成资产、端口和版本记录。

验收产物

- 网络拓扑；
- IP 规划；
- 存储挂载；
- 节点资产表。

第 35 周：驱动、CUDA、NCCL、K8s / Slurm、DCGM

视频

- 主视频: [NVIDIA GPU Operator Architecture](#)
- 补充: [DCGM](#), [Prometheus](#), [Grafana GPU Monitoring](#)
- 补充: [NCCL Explained](#)
- 官方: [GPU Operator](#)、[DCGM](#)、[NCCL](#)

操作步骤

1. 安装 Driver、CUDA、NCCL、Container Toolkit。
2. 安装 GPU Operator、Network Operator、DCGM。
3. 验证 `nvidia-smi`、`nvidia-smi topo -m`。
4. 执行 `dcmi diag -r 2`。
5. 运行 `nvbandwidth`、`nccl-tests`。
6. 加入 K8s 或 Slurm。
7. 配置 GPU、网络、存储和调度监控。

验收产物

- 健康检查报告；
- NVLink / NCCL 数据；
- 调度接入证明。

第 36 周：Burn-in、故障演练、验收和调优

视频

- 主视频: [A100 GPU 服务器 Burn-in](#)
- 补充: [NVIDIA NCA AI Infrastructure and Operations](#)
- 官方: [NVIDIA XID Errors](#)、[GPU Debug Guidelines](#)

操作步骤

1. `gpu-burn` 运行 30 ~ 60 分钟。
2. 监控温度、功耗、ECC、XID、降频。
3. 做 NVLink、PCIe、NCCL、RDMA、存储测试。
4. 做单卡、单节点、单轨故障演练。

5. 执行 `cordon / drain / repair / validate / uncordon`。
6. 部署 vLLM，做推理压测。
7. 输出验收报告、Runbook、容量和成本报告。

验收产物

- Burn-in 报告；
- 故障演练报告；
- 集群验收报告；
- 运维 Runbook。

附录 A：视频失效后的检索公式

B 站：

主题 + 入门/实战/保姆级/原理
主题 + 源码/部署/调优/故障
主题 + vLLM/Kubernetes/NVIDIA

YouTube：

topic + tutorial
topic + full course
topic + hands-on
topic + production
topic + official

搜索后按以下顺序筛选：

1. 官方频道优先；
2. 发布时间较新；
3. 有完整章节目录；
4. 有代码仓库；
5. 评论中能确认命令可运行；
6. 最终必须回到官方文档验证。

附录 B：每门视频的学习记录模板

```
# Video Notes: <title>

- URL:
- Author / Channel:
- Date:
- Duration:
- Version/Environment:

## 5 key points
1.
2.
3.
4.
5.

## Commands I ran
## Results
## Errors
## What I still need to verify
## Link to official docs
```

附录 C：学习完成的最低标准

- [] 每个主视频都有一条自己的实验记录。
- [] 每个知识点至少有一条官方文档来源。
- [] 每个实验都有命令、结果、错误和结论。
- [] 模型服务有压测、监控、限流、回滚。
- [] Harness 有权限、审批、恢复、评测和 Trace。
- [] GPU 集群有资产、网络、存储、Burn-in、告警、Runbook。
- [] 毕业项目能从用户问题一路讲到 GPU 节点和成本。

第二部分：AI Harness 企业落地教程

来源文件：02_AI_Harness企业落地教程.md

AI Harness 企业落地教程

目标：从零实现一个模型可替换、工具可治理、状态可恢复、过程可观测、结果可评估的 Agent Harness，并把它推进到企业可试点状态。

1. Harness 到底是什么

微软 Agent Framework 将 Agent Harness 定义为“把语言模型变成能执行工作的 Agent 的运行时脚手架”：它驱动模型和工具调用，管理会话状态和上下文，执行审批策略，并让 Agent 能持续推进多步任务。

用工程语言说：

```
Harness = Runtime Scaffolding around Model
          = Context + Memory + Planning + Tools + Policy
            + Sandbox + Verification + State + Observability + Evaluation
```

模型只负责推理和生成；Harness 负责让推理变成受控、可追踪、可恢复的执行。

Harness 不等于什么

概念	区别
Prompt Engineering	只优化输入文本；Harness 还管理工具、状态、权限、恢复和评估
Agent Framework	提供 Building Blocks；Harness 还定义团队运行规约和企业控制面
Workflow Engine	主要执行固定 DAG；Harness 允许模型动态决定下一步，但必须设置边界
MCP	工具和数据接入协议；MCP 是 Harness 的一个工具层，不是 Harness 本身
Eval Harness	常用于模型 Benchmark；这里的 Harness 指 Agent 运行时

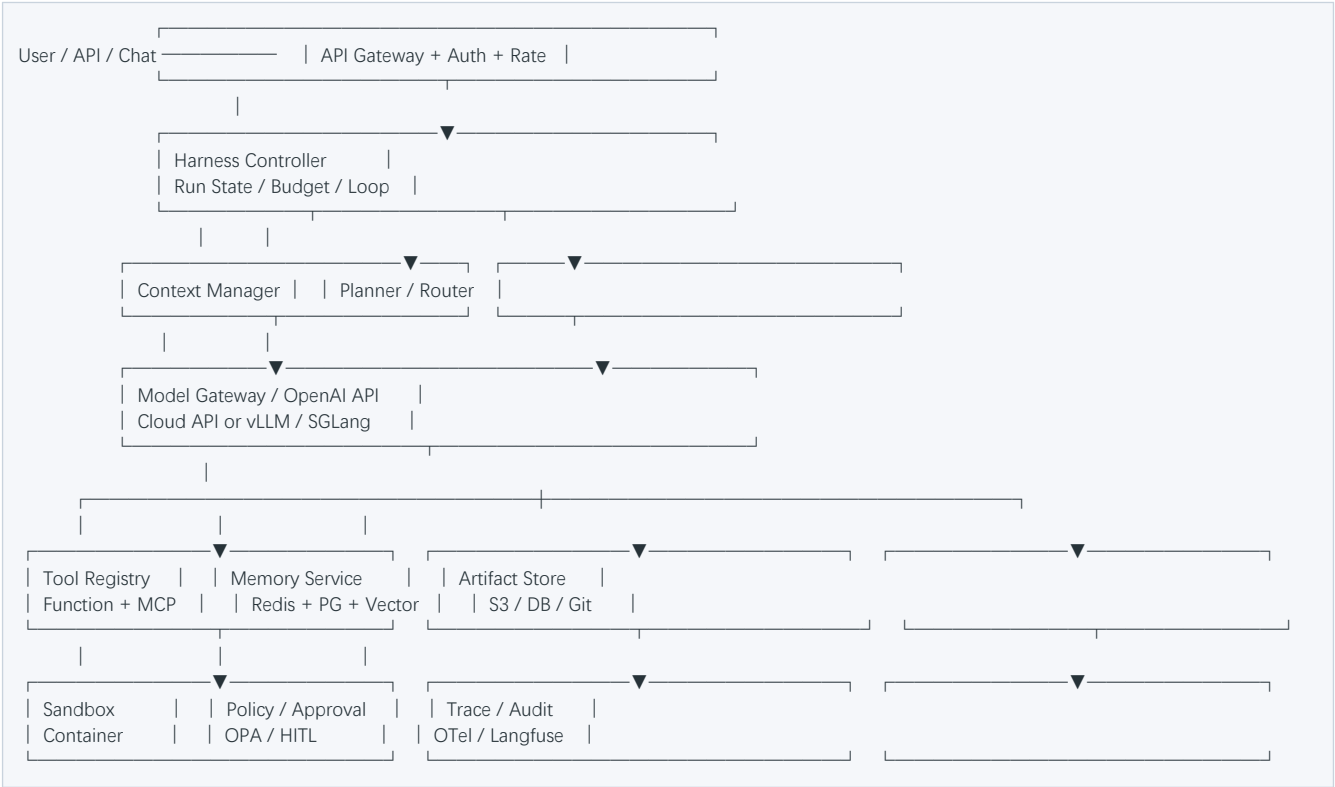
企业为什么需要自建 Harness

BCG 2026 年文章提出一个关键判断：模型可以采购，但企业自己的流程逻辑、质量标准、治理要求和组织知识，需要通过 Harness 固化，才会形成差异化能力。

建议遵循四个原则：

- 1. **Start small**：先选一个文档化、可验证、已有工具链的工作流试点。
- 2. **Build, don't buy**：模型和通用框架可以采购，企业控制面和领域适配要自己掌握。
- 3. **Be selective**：不是所有流程都适合 Agent；优先选择输入输出明确、质量可检查、工具可复用的流程。
- 4. **Model-agnostic**：业务逻辑不要和某一个模型或一家供应商强绑定。

2. 参考架构



核心组件

组件	最小职责	企业增强
Model Gateway	统一模型调用、超时、重试、Token 统计	模型路由、降级、成本预算、租户配额
Context Manager	构造每轮 Prompt，裁剪和摘要历史	分层上下文、压缩、敏感字段过滤
Planner / Router	决定下一步、调用哪个模型或工具	Plan-and-Execute、Critic、Subagent
Tool Registry	注册工具、JSON Schema、版本、权限	MCP、A2A、审批、幂等、审计
Run State	保存任务、步骤、预算、错误、产物	可恢复、可取消、可重放、事件溯源
Memory	短期消息、摘要、事实、产物索引	Redis + PostgreSQL + pgvector / Milvus
Sandbox	隔离代码和文件操作	容器、gVisor、网络白名单、资源限制
Policy / Approval	哪些工具能调用，何时需要人工确认	OPA / Kyverno、RBAC、数据分级、双人审批
Verification	测试、Schema、引用、Critic、二次采样	业务规则、人工复核、自动回滚
Observability	记录每次模型和工具调用	OTel GenAI、Langfuse、成本和失败归因
Evaluation	固定评测集和回归测试	离线评测、线上监控、红队、A/B

3. 建议项目结构

```
agent-harness/
├── pyproject.toml
├── .env.example
├── AGENTS.md
├── apps/
│   ├── api/
│   └── worker/
├── harness/
│   ├── controller.py
│   ├── context.py
│   ├── memory.py
│   ├── model_gateway.py
│   ├── planner.py
│   ├── policy.py
│   ├── sandbox.py
│   ├── verifier.py
│   └── tracing.py
├── tools/
│   ├── registry.py
│   ├── read_file.py
│   ├── search_knowledge.py
│   └── mcp_servers/
├── schemas/
│   ├── agent_spec.py
│   ├── run_state.py
│   └── tool_spec.py
├── evals/
│   ├── datasets/
│   └── suites/
├── tests/
│   ├── unit/
│   ├── integration/
│   └── adversarial/
├── deploy/
│   ├── docker-compose.yml
│   └── k8s/
└── reports/
```

环境初始化

```
uv init agent-harness
cd agent-harness
uv add fastapi uvicorn pydantic pydantic-settings httpx openai sqlalchemy psycpg redis opentelemetry-sdk
uv add --dev pytest pytest-asyncio ruff mypy
```

如果模型后端使用自部署 vLLM，只需要提供 OpenAI 兼容地址：

```
export OPENAI_BASE_URL=http://localhost:8000/v1
export OPENAI_API_KEY=dummy
export MODEL_NAME=Qwen/Qwen2.5-7B-Instruct
```

4. 八个递进实验

实验 1：最小 Harness，先让循环可控

目标：实现“输入任务 → 模型 → 工具调用 → 结果回填 → 最终回答”的最小闭环。

必须有的限制：

- 最大步骤数；
- 最大运行时间；
- 最大 Token / 成本预算；
- 模型调用超时；

- 工具调用超时；
- 不允许无限重试。

核心数据模型示意：

```
from datetime import datetime
from typing import Any, Literal
from pydantic import BaseModel, Field

class AgentSpec(BaseModel):
    name: str
    instructions: str
    model: str
    max_steps: int = 8
    max_seconds: int = 120
    allowed_tools: list[str] = Field(default_factory=list)

class RunState(BaseModel):
    run_id: str
    tenant_id: str
    user_id: str
    goal: str
    status: Literal["running", "waiting_approval", "succeeded", "failed", "cancelled"]
    step: int = 0
    messages: list[dict[str, Any]] = Field(default_factory=list)
    artifacts: list[dict[str, Any]] = Field(default_factory=list)
    error: str | None = None
    created_at: datetime
    updated_at: datetime
```

最小控制器伪代码：

```
async def run_harness(spec: AgentSpec, state: RunState, tools: ToolRegistry):
    while state.step < spec.max_steps and state.status == "running":
        state.step += 1

        context = build_context(state)
        response = await model_gateway.chat(
            model=spec.model,
            messages=context,
            tools=tools.schemas_for(spec.allowed_tools),
            timeout=30,
        )

        state.messages.append(response.as_message())

        if not response.tool_calls:
            state.status = "succeeded"
            break

        for call in response.tool_calls:
            policy.check(state, call, spec)
            result = await tools.execute(call, timeout=30)
            state.messages.append(tool_result_message(call, result))

    if state.step >= spec.max_steps:
        state.status = "failed"
        state.error = "max_steps_exceeded"

    return state
```

测评指标：

- 任务完成率；
- 平均步骤数；
- 工具调用成功率；
- 超时率；

- 平均 Token 和成本。

交付物：controller.py、schemas/、10 个正常用例、5 个失败用例、reports/lab1.md。

实验 2：Context 与 Memory

目标：区分“当前上下文”和“长期记忆”，防止把所有历史原样塞入 Prompt。

采用四层记忆：

层	内容	存储
Working Memory	当前任务最近几轮消息	Redis / 进程内
Session Summary	会话摘要、关键决定、未完成任务	PostgreSQL
Long-term Facts	用户偏好、组织事实、稳定知识	PostgreSQL + pgvector
Artifacts	文件、报告、代码、表格	S3 / 对象存储 + 索引

压缩策略：

1. 保留系统指令和当前目标；
2. 保留最近 N 轮完整消息；
3. 对旧消息生成结构化摘要；
4. 按当前问题检索长期记忆；
5. 只注入相关事实和证据；
6. 对敏感字段做脱敏或禁止写入。

验收：

- 100 轮对话后仍能回答 10 个关键事实；
- 历史压缩后不丢失当前目标和待办；
- 用户 A 的记忆绝不泄露给用户 B；
- 记忆有来源、时间、置信度和删除机制。

实验 3：Tool Registry 与 MCP

目标：让工具可发现、可版本化、可治理。

工具必须声明：

```
class ToolSpec(BaseModel):
    name: str
    version: str
    description: str
    risk: Literal["read", "write", "destructive"]
    timeout_seconds: int
    idempotency_required: bool
    required_scopes: list[str]
    input_schema: dict
    output_schema: dict
```

工具设计原则：

- 工具名称使用稳定命名空间，例如 crm.customer.get；
- 输入输出使用 JSON Schema；
- 返回结果要压缩、结构化、带来源和错误码；
- 写操作必须支持幂等键；
- 危险操作必须经过审批；
- MCP Server 只暴露最小必要能力；
- 工具调用结果和参数写入审计日志。

MCP 接入步骤：

- 1. 用官方 SDK 创建最小 MCP Server；
- 2. 先实现只读的 `search_knowledge` 和 `get_ticket`；
- 3. 在 Harness 中做 MCP Client；
- 4. 给每个工具设置超时、重试和并发上限；
- 5. 把工具 schema、版本、权限和审计接入 Tool Registry。

示意代码：

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("enterprise-tools")

@mcp.tool()
def get_ticket(ticket_id: str) -> dict:
    """Get ticket by ID. Read-only operation."""
    return ticket_service.get(ticket_id)

if __name__ == "__main__":
    mcp.run()
```

具体包名和启动方式以当前 MCP 官方 SDK 版本为准。

验收：工具调用失败能返回稳定错误码；没有权限的工具不会被模型看到；写操作没有幂等键时被拒绝。

实验 4：Sandbox 与 Human-in-the-Loop

目标：把“能执行”与“有权执行”分开。

沙箱要求：

- 文件系统只挂载允许目录；
- 默认禁止公网，按域名白名单放行；
- CPU、内存、磁盘、时间有上限；
- 子进程受控；
- 容器用户非 root；
- 产物输出到隔离目录；
- 记录命令、退出码和资源消耗。

审批矩阵：

操作	默认策略
只读检索	可自动执行
创建草稿、生成报告	可自动执行，但需日志
发邮件、写工单、修改数据	人工审批
删除数据、转账、发布生产	双人审批或禁止 Agent 执行

审批状态机：

```
RUNNING -> WAITING_APPROVAL -> APPROVED -> RUNNING
        -> REJECTED -> FAILED
        -> TIMEOUT -> FAILED
```

验收：模型不能通过 Prompt 绕过工具权限；审批有用户、时间、理由、参数快照和审计记录。

实验 5：Verification Loop

目标：Agent 不是“生成完就算结束”，而是要通过验证。

可选验证器：

- JSON Schema 校验;
- 单元测试;
- 静态检查 / lint;
- SQL 只读校验;
- 引用来源存在性检查;
- 事实一致性检查;
- Critic Model 评审;
- 业务规则引擎;
- 人工采样复核。

循环规则:

```
Generate -> Verify -> Pass => Finish
          -> Fail => Repair once
          -> Fail again => Escalate / Stop
```

验收: 每个任务至少有一个可机器判断的通过条件; 不能通过验证时不允许标记成功。

实验 6: 长任务与跨 Session 恢复

Anthropic 的长任务经验指出: 仅有 Context Compaction 不够。更可靠的做法是:

1. 第一次运行使用 **Initializer Agent** 初始化项目、目录、验收标准和进度文件;
2. 后续每次运行使用 **Coding / Worker Agent** 做增量工作;
3. 结束时更新 `progress.md`、测试结果和 Git 历史;
4. 下一次从文件系统、数据库和版本控制恢复, 而不是依赖模型记忆。

状态必须持久化:

- 目标;
- 已完成步骤;
- 未完成步骤;
- 关键决定;
- 已知失败;
- 产物位置;
- 下一步建议;
- 最后更新时间。

交付物: 模拟 5 次上下文重置, 任务仍能从 `run_state + artifacts + progress.md` 恢复。

实验 7: Observability 与 Evaluation

Trace 至少记录:

- `run_id`、`trace_id`、租户、用户;
- 模型、Prompt 版本、Token、成本;
- 每次工具调用的名称、参数摘要、耗时、结果状态;
- 检索结果和引用来源;
- 审批和人工干预;
- 最终结果和用户反馈。

评测层次:

层次	例子
组件测试	工具 schema、超时、重试、幂等
路径测试	ReAct 是否按预期调用工具
任务评测	20 ~ 100 个真实任务的成功率
安全评测	Prompt Injection、越权、数据外泄
线上评测	成功率、人工接管率、成本、延迟、用户满意度

工具：

- OpenTelemetry GenAI Semantic Conventions：统一 Trace 字段；
- Langfuse：Trace、Prompt、成本、评估；
- MLflow：实验、评测、Prompt 和模型生命周期；
- promptfoo / Ragas / DeepEval：自动回归和红队。

验收：任意一个失败 Run 都能在 5 分钟内定位到“模型、上下文、工具、权限、数据、网络”中的哪一层。

实验 8：企业部署与治理

部署拓扑：

API Gateway
-> Harness API
-> Run Queue
-> Harness Workers
-> Model Gateway
-> Tool / MCP Services
-> Memory / Artifact / Audit

生产要求：

- 多租户隔离；
- 队列和并发限制；
- Run 可取消、可恢复、可重放；
- 模型路由和降级；
- 预算和 Token 配额；
- 密钥托管；
- 工具版本化；
- Prompt 和 Skill 版本化；
- CI/CD 与灰度；
- 回滚和 Runbook；
- 日志脱敏；
- 数据保留和删除策略。

企业角色分工：

角色	责任
Platform Team	Harness Runtime、Gateway、Memory、Tool Registry、Observability
Domain Team	领域工具、知识库、评测集、业务流程
Security	权限、数据分类、沙箱、审计、红队
SRE	SLO、容量、故障恢复、成本
Product / PDE	用户问题、交互、指标、上线迭代

5. AGENTS.md 与 Skills 最小模板

AGENTS.md

```
# AGENTS.md

## Repository expectations
- Run `uv run pytest` and `uv run ruff check .` before proposing a change.
- Never commit secrets or customer data.
- Update `docs/architecture.md` when changing interfaces.

## Safety rules
- Do not run destructive shell commands without explicit approval.
- Use read-only database credentials for analysis.
- Store generated artifacts under `reports/` or `artifacts/`.

## Code Review Rules
- Flag any tool call that writes data without an idempotency key.
- Flag any prompt that includes raw PII or secrets.
```

Skill 模板

```
---
name: incident-triage
description: Use when triaging a production incident with logs, metrics, and known runbooks.
---

1. Collect incident metadata and time range.
2. Query metrics before logs.
3. Identify the failing dependency.
4. Propose the smallest safe mitigation.
5. Ask for human approval before changing production.
6. Produce a post-incident summary and follow-up tasks.
```

6. Harness 安全基线

至少覆盖 OWASP LLM Top 10 和 NIST AI RMF 中的相关风险：

- Prompt Injection：把外部文档视为不可信输入；
- 敏感信息泄露：输入、输出、日志、Trace、Memory 全链路脱敏；
- 过度代理：限制工具、步骤、预算和写操作；
- 供应链风险：锁定模型、库、MCP Server 和容器版本；
- 不当输出处理：输出先校验再进入下游系统；
- 无界消费：Token、请求、并发和成本预算；
- 工具滥用：最小权限、审批、幂等和审计。

7. 完成标准

- [] 能解释 Harness 与 Framework、Workflow、MCP、Prompt Engineering 的区别。
- [] 能画出一张完整的 Harness 架构图。
- [] 有最大步骤、时间、Token 和成本限制。
- [] 工具有 Schema、版本、风险级别、权限和审计。
- [] 写操作有幂等和人工审批。
- [] 运行状态可持久化、取消、恢复和重放。
- [] 有 Trace 和一条从用户请求到最终结果的完整链路。
- [] 有正常、失败、对抗和越权评测集。
- [] 能通过模型替换验证 Harness 不与单一供应商绑定。

[] 完成一次 5 个 Session 的中断恢复实验。

第三部分：AI PDE 产品工程落地教程

来源文件：03_AI_PDE产品工程落地教程.md

AI PDE 产品工程落地教程

本文的 PDE = **Product Design / Product Development Engineer**：端到端负责用户问题、交互设计、工程实现、AI 评估和上线迭代。

如果你的 PDE 指 Partial Differential Equations，请使用 AI for Science 路线，不适用本教程。

1. PDE 在企业里到底负责什么

AI PDE 不是“会用 AI 画页面的人”，而是能把下面四件事串起来的人：

用户问题 -> 产品方案 -> 交互与原型 -> 生产实现 -> 评估与上线 -> 数据反馈 -> 再迭代	
能力	要能交付什么
产品判断	用户问题、业务价值、优先级、成功指标
交互设计	用户流程、信息架构、状态机、失败恢复、可访问性
全栈工程	前端、API、数据契约、模型 / Harness 接入、部署
AI 评估	离线评测集、任务成功率、幻觉 / 引用、回归测试
上线迭代	Feature Flag、灰度、监控、A/B、回滚、复盘

PDE 最适合 AI 产品的原因：AI 产品的关键状态很多，不可能只靠一张静态稿描述。

必须设计的 AI 状态至少包括：

idle
understanding
retrieving
planning
streaming
waiting_for_tool
waiting_for_approval
partial_result
needs_clarification
failed_and_retryable
failed_terminal
completed_with_citations
completed_with_low_confidence

2. PDE workflow

阶段 1：Discover，先确认值不值得做

输入：

- 用户访谈；
- 现有流程和工单；
- 数据基线；
- 风险和合规边界。

输出：

- JTBD / 用户故事；
- 当前流程和痛点；
- 成功指标；

- 明确不做什么；
- 失败和退出条件。

阶段 2：Spec，写一个 Agent 能执行的规格

传统 PRD 只描述功能，AI 产品要额外写清：

- 输入边界；
- 输出格式；
- 数据来源；
- 工具权限；
- 不确定性处理；
- 何时澄清；
- 何时拒答；
- 何时人工接管；
- 如何评估。

推荐规格结构：

```
# Feature Spec

## User problem
## Target user
## Job to be done
## Current workflow
## Proposed AI workflow
## Tool and data boundaries
## UX states
## API contract
## Evaluation rubric
## SLO
## Security and privacy
## Rollout and rollback
```

阶段 3：Prototype，用真实延迟和真实数据验证

不要只做静态高保真图。原型至少验证：

- 流式输出是否可理解；
- 长时间等待是否有反馈；
- 引用来源是否可核查；
- 工具失败如何恢复；
- 审批是否打断得太频繁；
- 用户是否知道 AI 当前在做什么。

阶段 4：Build，生产实现

优先技术栈：

- 前端：Next.js / React / TypeScript；
- 后端：FastAPI / Pydantic；
- 流式：SSE 或 WebSocket；
- 模型：统一走 Harness Model Gateway；
- 数据：Postgres + pgvector / Redis / 对象存储；
- 监控：OpenTelemetry + 产品事件；
- 部署：Docker，后续接到 Kubernetes。

阶段 5: Evaluate，先证明不是 Demo

三层评估：

层	指标
AI 层	任务完成率、引用正确率、工具正确率、拒答准确率
产品层	完成时间、接管率、重试率、满意度、留存
业务层	节省工时、降低错误率、收入 / 成本、合规事件

阶段 6: Ship，控制风险

- Feature Flag 和灰度；
- 线上评测集和人工抽样；
- 监控延迟、错误率、成本和投诉；
- Runbook 和降级方案；
- 一键关闭和回滚；
- 上线后 1 ~ 2 周复盘。

3. 七个递进实验

实验 1：用户问题与基线

任务：选择一个真实流程，例如：

- 企业知识问答；
- 客服工单分类与建议；
- 运维故障初筛；
- 销售资料整理；
- 数据分析自然语言查询。

必须完成：

1. 访谈 3 位目标用户；
2. 记录当前流程的步骤、耗时和错误；
3. 写出 JTBD；
4. 选定 1 个主指标和 2 个护栏指标；
5. 明确 AI 不做的部分。

产出：product/discovery.md、用户旅程、基线数据。

实验 2：AI PRD 与状态机

任务：把需求写成 Agent 可执行 Spec。

必须包含：

- 主流程和异常流程；
- 工具与数据权限；
- 澄清 / 拒答 / 接管条件；
- 输入输出 JSON；
- 评测样例；
- SLO：P50/P95 延迟、成功率、成本上限。

产出：product/prd.md、Mermaid 状态机、API 契约。

示意：

```
stateDiagram-v2
    [*] --> Understanding
    Understanding --> Clarifying: 信息不足
    Clarifying --> Understanding: 用户补充
    Understanding --> Retrieving: 需要知识
    Retrieving --> Planning: 检索完成
    Planning --> WaitingApproval: 需要写操作
    WaitingApproval --> Executing: 审批通过
    WaitingApproval --> Failed: 拒绝 / 超时
    Planning --> Executing: 只读操作
    Executing --> Verifying
    Verifying --> Completed: 通过
    Verifying --> Repairing: 失败但可修复
    Repairing --> Verifying
    Verifying --> ManualHandoff: 连续失败
    Completed --> [*]
    ManualHandoff --> [*]
    Failed --> [*]
```

实验 3：可交互原型

初始化前端：

```
pnpm create next-app@latest ai-pde-ui --typescript --tailwind --eslint --app --src-dir
cd ai-pde-ui
pnpm dev
```

初始化后端：

```
uv init ai-pde-api
cd ai-pde-api
uv add fastapi uvicorn pydantic httpx python-multipart
uv run uvicorn app.main:app --reload --port 8080
```

原型必须包含：

- 流式输出；
- 停止生成；
- 重新生成；
- 引用卡片；
- 工具运行状态；
- 失败与重试；
- 人工审批；
- 反馈按钮；
- “不确定 / 信息不足”的提示。

验收：用真实模型和真实知识库，不用写死 Mock 数据。

实验 4：接入 Harness 与工具

前端不直接调用模型，统一调用 Harness API：

```
POST /v1/runs
GET /v1/runs/{run_id}
POST /v1/runs/{run_id}/cancel
POST /v1/runs/{run_id}/approve
GET /v1/runs/{run_id}/events
```

原则：

- UI 只消费稳定的 Run API；

- 模型和推理引擎在后端替换；
- 工具调用展示为可理解的动作；
- 写操作显示影响范围和审批理由；
- 所有产物带来源和版本。

验收：替换模型后端不改前端业务代码。

实验 5：评测与回归

建立 evals/：

```
evals/
├── datasets/
│   ├── golden_questions.jsonl
│   ├── ambiguous_questions.jsonl
│   └── adversarial_prompts.jsonl
├── suites/
│   ├── task_success.yaml
│   ├── retrieval.yaml
│   └── safety.yaml
└── reports/
```

评测维度：

维度	问题
正确性	回答是否符合事实和任务目标
完整性	是否遗漏关键步骤
引用	是否支持每个关键结论
工具	是否调用正确工具和参数
安全	是否越权、泄露、被注入
体验	是否澄清、可恢复、可理解
成本	每个成功任务的花费是否可接受

验收：每次 Prompt、模型、工具、知识库变更都跑回归；报告能展示新旧版本差异。

实验 6：灰度、监控与用户反馈

产品事件最小集合：

```
run_started
run_succeeded
run_failed
run_cancelled
approval_requested
approval_granted
approval_rejected
tool_called
tool_failed
citation_clicked
feedback_submitted
manual_handoff
```

看板必须展示：

- 任务成功率和人工接管率；
- P50/P95/P99 延迟；
- 工具失败率；
- 单次成功成本；
- 引用点击率；
- 用户反馈；

- 按租户 / 模型的错误分布。

灰度策略：

1. Internal Dogfood;
 2. 1% 真实用户;
 3. 10% 流量并观察一周;
 4. 50%;
 5. 全量或回滚。
-

实验 7：上线交付与复盘

上线清单：

- PRD 和架构图;
- Runbook;
- Feature Flag;
- 权限和数据流图;
- 评测报告;
- 压测报告;
- 监控面板;
- 告警策略;
- 回滚步骤;
- 客服 / 运营培训;
- 反馈处理流程。

复盘问题：

- 哪个用户指标真正改变?
 - 哪些请求最容易被人工接管?
 - 哪些失败来自模型、工具、数据还是交互?
 - 哪个 Prompt / Skill / 工具版本最好?
 - 成本是否随使用量线性增长?
 - 下一步应该优化质量、速度还是成本?
-

4. AI PRD 模板

AI Feature Spec: <name>

1. Problem

- User:
- Pain:
- Current baseline:
- Why now:

2. Goal and guardrails

- Primary metric:
- Guardrail metrics:
- Non-goals:

3. User workflow

- Entry point:
- Happy path:
- Ambiguous path:
- Failure path:
- Human handoff:

4. AI design

- Model / Harness:
- Tools:
- Data sources:
- Memory:
- Context limits:
- Citation policy:
- Abstention policy:

5. API contract

- Request:
- Response:
- Streaming events:
- Error codes:

6. Evaluation

- Golden set:
- Adversarial set:
- Acceptance threshold:
- Regression gate:

7. SLO and cost

- P95 latency:
- Availability:
- Cost per successful task:
- Token budget:

8. Security and privacy

- Data classification:
- Access control:
- Retention:
- Audit:
- Injection defense:

9. Rollout

- Flag:
- Canary:
- Rollback:
- Owner:

5. PDE 与其他角色的协作边界

角色	主要责任	PDE 交付给谁
Product Manager	业务优先级、目标、商业指标	可执行 Spec、原型、上线结果
Designer	交互、视觉、Design System	用户流程、状态、组件规范
AI Engineer	模型、RAG、评测、Agent 逻辑	稳定 API、评测要求、业务规则
Infra Engineer	推理平台、GPU、K8s、SLO	资源需求、延迟、容量和降级要求
Security	权限、数据、供应链和合规	数据流、工具风险、审计需求
Data	数据质量、指标、实验	事件定义、评测数据、分析结果

PDE 的价值不是替所有人工作，而是减少交接损耗，并对“从问题到生产结果”负责。

6. 完成标准

- [] 有真实用户问题和基线，不是拍脑袋需求。
- [] 有完整 PRD、状态机和 API 契约。
- [] 原型包含流式、等待、失败、重试、审批和不确定性状态。
- [] 前端不直接绑定模型供应商。
- [] 有 Golden Set、对抗集和回归门禁。
- [] 有任务成功、接管率、延迟、成本和满意度指标。
- [] 能灰度、监控、回滚和人工接管。
- [] 能解释一个失败案例属于模型、数据、工具、权限还是交互问题。
- [] 至少完成一次上线后复盘，并形成下一版迭代计划。

第四部分：AI Infra 企业平台教程

来源文件：04_AI_Infra企业平台教程.md

AI Infra 企业平台落地教程

目标：从单机 vLLM 出发，构建一套可支撑多团队、多模型、多租户的 AI 推理平台基础，并理解从单机到多机分布式、从模型部署到平台治理的完整路径。

1. AI Infra 的边界

AI Infra 不等于“会启动 vLLM”。企业 AI Infra 至少包含九层：

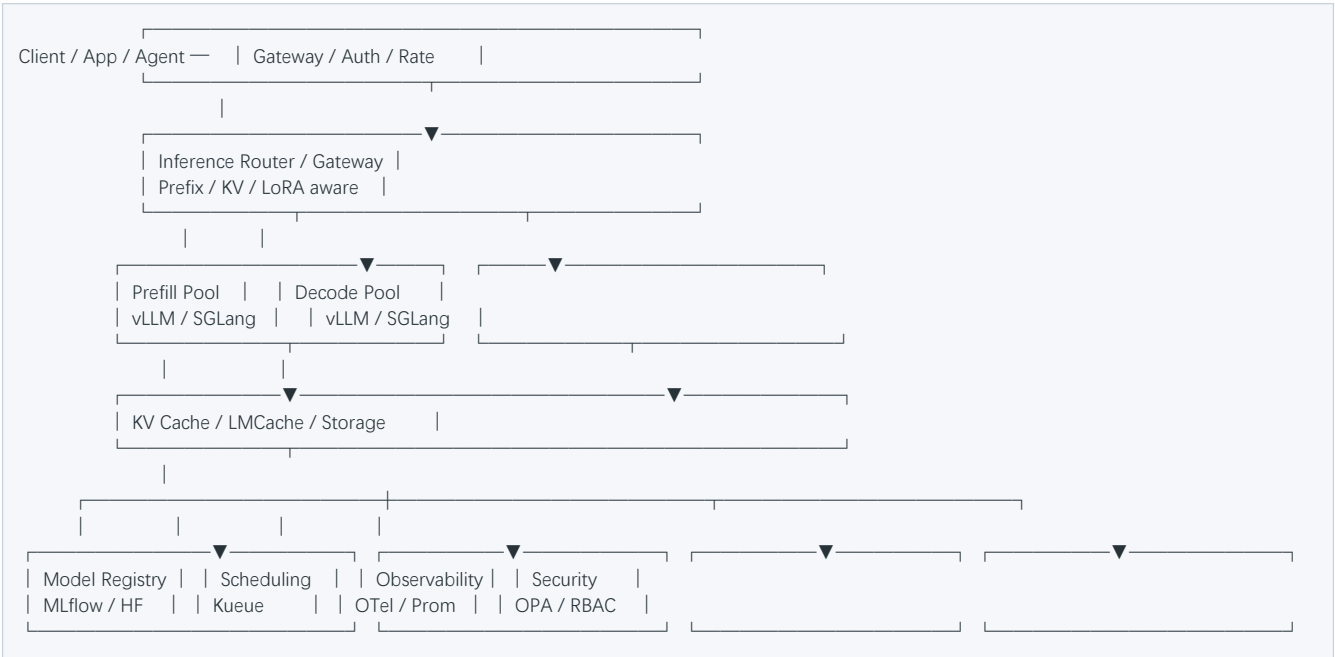
1. 芯片与节点	GPU / NPU / CPU / RDMA / 存储
2. 驱动与运行时	Driver / CUDA / NCCL / GPU Operator
3. 推理引擎	vLLM / SGLang / TensorRT-LLM / NIM
4. 服务与编排	Ray Serve / KServe / IIm-d / Dynamo
5. 路由与网关	Gateway API / KV-aware Router / LiteLLM
6. 调度与容量	Kueue / 队列 / 配额 / 优先级 / 抢占
7. 数据与状态	Object Store / Model Registry / Redis / Vector DB
8. 观测与评估	OTel GenAI / Prometheus / Grafana / MLflow / Langfuse
9. 安全与治理	RBAC / OPA / 沙箱 / DLP / 审计 / 成本

企业成熟度模型

等级	形态	特征
L1	单机脚本	手工启动 vLLM，无监控，无模型版本
L2	单机生产服务	Docker、网关、健康检查、指标、日志、压测
L3	K8s 单集群	GPU Operator、多副本、自动扩缩容、模型缓存
L4	分布式推理平台	TP / PP、KV-aware 路由、Prefill/Decode、Dynamo/IIm-d
L5	多租户 AI 平台	配额、抢占、评测、安全、成本、审计、自服务

学习顺序必须是 L1 → L2 → L3 → L4 → L5，不要一开始就上最高等级。

2. 参考架构



最小组件选择

目标	推荐起点	升级方向
模型推理	vLLM	SGLang / TensorRT-LLM / NIM
K8s 服务	Ray Serve / KServe	llm-d / Dynamo
路由	Nginx / LiteLLM	Gateway API Inference Extension
调度	Kubernetes	Kueue 配额和抢占
监控	Prometheus + Grafana	OTel GenAI + Langfuse
模型管理	HuggingFace + Git	MLflow / 企业模型仓库
评估	promptfoo / Ragas	在线评测和业务指标

3. 十三个递进实验

实验 0：环境和 GPU 基线

检查：

```
nvidia-smi
nvidia-smi topo -m
nvcc --version
python --version
uv --version
docker version
kubectl version --client
```

记录：

- GPU 型号、显存、驱动、CUDA；
- GPU 互联是 NVLink、PCIe 还是跨机网络；
- CPU、内存、本地盘、对象存储；
- 网络带宽和 RDMA 能力；
- 单卡和各卡组合的理论显存。

验收：有一份可复现的 `environment.md`，不是只截一张 `nvidia-smi` 图。

实验 1：vLLM 单机服务与 OpenAI API

```
uv venv --python 3.12
source .venv/bin/activate
uv pip install vllm

vllm serve Qwen/Qwen2.5-7B-Instruct \
  --served-model-name qwen2.5-7b \
  --gpu-memory-utilization 0.90 \
  --max-model-len 4096 \
  --port 8000
```

调用：

```
curl http://localhost:8000/v1/chat/completions \
  -H 'Content-Type: application/json' \
  -d '{
    "model": "qwen2.5-7b",
    "messages": [{"role": "user", "content": "解释 KV Cache"}],
    "stream": true
  }'
```

必须理解：

- Prefill 和 Decode；
- PagedAttention；

- Continuous Batching;
- Prefix Cache;
- gpu-memory-utilization 与 max-model-len。

验收：

- 流式和非流式均可用；
- 记录 TTFT、输出 token/s、QPS、显存；
- 能解释 OOM 时先改什么、为什么。

实验 2：压测与性能基线

推荐工具：

- vllm bench serve;
- GuideLLM;
- 自建 asyncio 压测脚本。

至少测试：

并发：1, 4, 8, 16, 32
输入：短、中、长
输出：短、中、长
缓存：Prefix Cache 开 / 关

指标：

指标	含义
TTFT	首 Token 延迟
TPOT / ITL	每输出 Token 时间 / Token 间隔
Output token/s	输出吞吐
Request throughput	请求吞吐
Queue time	请求排队时间
GPU memory used	显存使用
KV cache usage	KV Cache 占用
Error rate	错误率

交付物：benchmarks/ 原始数据、图表和结论。

实验 3：Docker 化与最小生产服务

要求：

- 固定镜像版本；
- 非 root 用户；
- 模型只读挂载；
- 健康检查；
- 结构化日志；
- Secrets 通过环境变量或 Secret 注入；
- 容器内只暴露必要端口；
- 有资源 request / limit。

本地启动示意：

```
docker build -t enterprise-vllm:0.1.0 .
docker run --rm --gpus all \
-p 8000:8000 \
--env-file .env \
-v /models:/models:ro \
enterprise-vllm:0.1.0
```

验收：镜像可重复构建，模型、代码、配置和镜像都有版本号。

实验 4：模型网关与统一路由

目标：应用不直接依赖某个 vLLM 实例，而是通过统一网关访问模型。

网关职责：

- 统一 OpenAI API；
- 多模型路由；
- API Key / 租户身份；
- 速率限制和配额；
- 超时、重试和熔断；
- Token / 成本统计；
- 模型版本和灰度；
- 敏感词或安全策略接入。

可选：

- [LiteLLM](#)；
- [Gateway API Inference Extension](#)；
- 企业 API Gateway + 自研适配层。

验收：替换后端模型或切到另一个副本，不需要修改业务代码。

实验 5：Kubernetes + GPU Operator

环境选择：

- 本地学习：kind / k3d（无真实 GPU 调度时只验证控制面）；
- 生产演练：带 GPU 的托管 Kubernetes 或自建 K8s；
- GPU 节点必须安装合适的驱动和容器运行时。

安装顺序：

1. Kubernetes；
2. GPU Operator；
3. 节点标签与污点；
4. GPU 资源验证；
5. 部署 vLLM；
6. Service / Ingress / Gateway；
7. 监控和日志。

检查：

```
kubectl get nodes
kubectl describe node <gpu-node>
kubectl get pods -n gpu-operator
kubectl get pods -A
```

验收：Pod 能看到 [nvidia.com/gpu](#) 资源，调度不会把 GPU 任务放到普通节点。

实验 6：部署 vLLM 到 Kubernetes

最小 Deployment 示意（生产必须固定镜像和模型版本）：

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm-qwen
spec:
  replicas: 1
  selector:
    matchLabels:
      app: vllm-qwen
  template:
    metadata:
      labels:
        app: vllm-qwen
    spec:
      containers:
        - name: vllm
          image: vllm/vllm-openai:<pinned-version>
          args:
            - Qwen/Qwen2.5-7B-Instruct
            - --served-model-name
            - qwen2.5-7b
            - --host
            - 0.0.0.0
            - --port
            - "8000"
            - --gpu-memory-utilization
            - "0.90"
          ports:
            - containerPort: 8000
      resources:
        limits:
          nvidia.com/gpu: 1
      readinessProbe:
        httpGet:
          path: /health
          port: 8000
        initialDelaySeconds: 30
        periodSeconds: 10
```

需要配套：

- Service;
- Secret;
- PVC / 模型缓存;
- PodDisruptionBudget;
- HPA / KEDA / KServe autoscaler;
- NetworkPolicy;
- ServiceMonitor。

验收：能滚动升级、回滚和优雅停止。

实验 7：Ray Serve / KServe / llm-d 选型

方案	适合	特点
Ray Serve LLM	Python 团队、复合 Agent / RAG 服务	组合 Deployment、扩缩容、Python 生态
KServe	企业 Kubernetes 标准化平台	CRD、Generative/Predictive、Canary、模型缓存
llm-d	大规模 K8s LLM 推理	Well-Lit Paths、路由、缓存、Prefill/Decode
vLLM Production Stack	以 vLLM 为中心的参考栈	Helm、路由、KV Cache Offloading、监控
NVIDIA Dynamo	超大规模分布式推理	多引擎、KV-aware Router、Disaggregated Serving

任务：选两个方案做同一 workload 的 PoC，不从宣传 Benchmark 直接得出结论。

验收：输出选型矩阵：功能、运维、成本、生态、可移植性、社区成熟度。

实验 8：Kueue 队列、配额与抢占

Kueue 是 Kubernetes 原生的作业排队和资源治理系统，支持：

- 多租户配额；
- 动态容量共享；
- 拓扑感知放置；
- Spot / On-Demand 资源池；
- 优先级与预抢占。

实验场景：

1. 创建两个团队队列；
2. 每队有 GPU 配额；
3. 提交高优先级推理 / 批处理作业；
4. 验证低优先级任务被挂起或抢占；
5. 记录等待时间、利用率和公平性。

验收：不是“能跑”，而是能解释谁先跑、为什么、是否会饿死。

实验 9：分布式推理 TP / PP / DP

策略	做什么	适用
TP	切层内矩阵，多卡共同完成一层	单机多卡、NVLink
PP	按层分段，多机接力	单机放不下、跨机通信
DP	多个完整副本，分发请求	提高吞吐和可用性
EP	MoE 专家分布	MoE 大模型

vLLM 示例：

```
vllm serve Qwen/Qwen2.5-14B-Instruct \
  --tensor-parallel-size 2 \
  --distributed-executor-backend mp \
  --gpu-memory-utilization 0.90 \
  --max-model-len 4096
```

必须测量：

- 单卡基线；
- TP=2；
- PP=2；
- TP+PP；
- 加速比、扩展效率、TTFT、吞吐、显存、通信时间。

验收：能解释为什么两张卡不等于两倍性能。

实验 10：KV-aware Routing、Prefill / Decode 分离

适合规模变大后再学：

- Prefix Cache 复用；
- KV Cache 感知路由；
- Prefill / Decode Disaggregation；
- KV Cache Offloading；
- LMCache；

- Disaggregated Serving。

参考：

- [NVIDIA Dynamo](#)；
- [llm-d](#)；
- [vLLM Production Stack](#)。

验收：能解释为什么路由不能只按 Round Robin；KV Cache 命中率和成本之间的关系是什么。

实验 11：观测、评估与模型生命周期

基础设施指标：

- GPU 利用率、显存、温度、功耗；
- 节点网络、RDMA、NCCL；
- Pod、队列、副本、重启；
- 请求延迟、错误、队列长度。

AI 指标：

- TTFT、TPOT、Token/s、成本 / 万 Token；
- KV Cache 使用率、Prefix 命中率；
- 工具调用成功率；
- 任务完成率、人工接管率；
- 引用正确率、幻觉率。

工具组合：

- OpenTelemetry GenAI：统一字段；
- Prometheus + Grafana：基础设施与服务；
- Langfuse / MLflow：Trace、Prompt、评测、模型版本；
- promptfoo / Ragas / DeepEval：质量回归。

验收：能从一次用户投诉追踪到模型、节点、请求、工具和版本。

实验 12：安全、可靠性与成本

安全：

- 身份与 RBAC；
- 租户隔离；
- 模型和工具权限；
- Secrets 管理；
- Prompt Injection 防护；
- 输入输出 DLP；
- 审计和保留策略；
- 供应链锁定。

可靠性：

- 多副本和反亲和；
- 健康检查；
- 超时、重试、熔断、降级；
- 模型加载失败处理；
- 节点故障和抢占；

- 备份与恢复；
- 混沌演练。

成本：

每百万输出 Token 成本 =
GPU 每小时成本 / 每小时输出 Token 数 × 1,000,000

必须把成本按租户、模型、业务线拆分，并设置预算告警。

4. 企业 SLO 模板

```
service: llm-inference
model: qwen2.5-7b
slo:
  availability: 99.9%
  ttft_p95_ms: 800
  tpot_p95_ms: 80
  error_rate: < 0.5%
  queue_time_p95_ms: 300
cost:
  max_per_million_output_tokens: <target>
security:
  tenant_isolation: true
  pii_logging: false
  tool_allowlist: true
```

SLO 必须有：

- Owner；
- 监控面板；
- 告警阈值；
- 降级策略；
- 复盘记录。

5. 完成标准

- [] 能在单机用 OpenAI 兼容 API 提供模型服务。
- [] 有可复现的 TTFT、Token/s、QPS 和显存压测报告。
- [] 能解释 vLLM 的 KV Cache、PagedAttention 和 Continuous Batching。
- [] 能把服务 Docker 化并固定版本。
- [] 能做到多副本和统一模型网关。
- [] 能在 K8s 上调度 GPU 并部署推理服务。
- [] 能配置队列、配额、优先级和抢占。
- [] 能完成 TP / PP 多卡实验并计算加速比。
- [] 能接入 Prometheus、Grafana 和 OpenTelemetry。
- [] 能完成安全、故障和成本演练。
- [] 能输出一份面向业务和运维的 AI Infra 架构图与 Runbook。

第五部分：GPU 物理集群建设、选型、部署与调优

来源文件：07_GPU物理集群建设选型部署与调优.md

GPU 物理集群建设、选型、部署与调优

范围： rack、power、cooling、GPU 节点、RDMA / InfiniBand / Ethernet、存储、管理网、驱动、CUDA、NCCL、调度、模型部署和性能调优。

原则： 先按 workload 和 SLO 选硬件，再设计网络、存储和调度；不要先买卡再想怎么用。

资料核验： **2026-10-04**。NVIDIA DGX SuperPOD 文档当前已覆盖 Rubin / Vera Rubin、GB300、B300、B200、H200 等多代基础设施，具体供货、功耗和兼容性必须按采购时官方资料确认。

1. GPU 物理集群的完整分层

应用层	Agent / RAG / 数据分析 / 训练任务
调度层	Slurm / Kubernetes / Kueue / Ray
服务层	vLLM / SGLang / TensorRT-LLM / NIM / Dynamo / llm-d
运行时层	CUDA / NCCL / MPI / RDMA / NVLink / GPUDirect
服务器层	GPU / CPU / HBM / PCIe / NUMA / BMC / PSU
网络层	计算网 / 存储网 / 管理网 / 带外网
存储层	模型库 / 数据集 / Checkpoint / 日志 / 对象存储
物理层	Rack / Power / Cooling / Fire / Security / Cabling

任何一层设计错误，都可能让上层模型性能、稳定性和成本失控。

2. 先写需求，再做硬件选型

2.1 必问问题

问题	影响
主要做训练、微调、推理还是混合？	决定 GPU 显存、互联、网络和存储
模型最大参数量、序列长度、并发？	决定单卡显存和 TP / PP / EP
延迟优先还是吞吐优先？	决定高主频、互联、批处理和副本策略
是否做多机？	决定 InfiniBand / RoCE、spine-leaf、rail-optimized
是否需要多租户？	决定 MIG、配额、Kueue、Slurm 账号体系
数据规模、Checkpoint 和读取模式？	决定并行文件系统、对象存储和缓存
可用性目标？	决定冗余网络、N+1 电源、备件和故障切换
功率、制冷、机柜和地板承重？	决定可部署规模
合规和地域？	决定是否私有化、国产化或专用云

2.2 选型决策树

模型能否放进单卡显存？
└─ 能：单卡或单机多副本
└─ 不能：能否放在单机 NVLink 域内？
└─ 能：单机多卡 TP / PP
└─ 不能：跨机 TP + PP / EP
└─ 训练 / 高频通信：InfiniBand / 高速 RoCE
└─ 推理 / 缓存友好：KV-aware routing + 分层缓存

3. GPU 与服务器选型

3.1 GPU 评估维度

维度	说明
显存容量	决定模型、KV Cache、Batch 和并发上限
HBM 带宽	决定 Decode 和内存密集任务速度
算力	FP16 / BF16 / FP8 / FP4 / INT8 的实际支持
互联	NVLink / NVSwitch 域大小和带宽
PCIe 带宽	与 NIC、存储和跨节点通信有关
MIG	是否需要安全、强隔离的小实例
ECC / 遥测	生产可靠性和故障定位能力
功耗与散热	风冷 / 液冷、机柜功率密度
软件生态	CUDA、NCCL、TensorRT、推理引擎支持
生命周期	供货、驱动、支持周期和备件

3.2 规模与硬件档位（规划启发式，不是采购规则）

规模	典型目标	设计重点
1 ~ 8 GPU	学习、PoC、单机推理	单机 NVLink、单机存储、Docker / K8s
8 ~ 32 GPU	部门级模型服务、微调	单/双机 NVLink、100/200/400G 网络、共享存储
32 ~ 256 GPU	企业推理集群、训练集群	spine-leaf、RDMA、多轨、并行文件系统、Kueue/ Slurm
256 ~ 1000+ GPU	AI Factory / SuperPOD 级	rail-optimized、液冷、DC busbar、N+1 电力、Mission Control / 平台软件

3.3 CPU、内存和本地盘

- CPU 核数与 PCIe Lane 要匹配 GPU 和 NIC 数量；
- CPU 内存至少考虑模型加载、Tokenization、数据预处理和 Checkpoint；
- 关注 NUMA 亲和，避免 GPU、NIC、CPU 跨 NUMA 访问；
- 本地 NVMe 用于模型缓存、临时 Checkpoint 和高速 scratch；
- 不要把长期数据集和唯一 Checkpoint 只放在本地盘。

4. 机柜、供电与散热

4.1 物理设计清单

- 机柜 U 数、深度、承重、前后风道；
- 单机柜功率、单路 PDU、供电冗余；
- UPS / 发电机 / ATS；
- 风冷还是液冷，CDU 容量和冗余；

- 漏液检测、温湿度、烟感和消防；
- 光纤、铜缆、DAC / AOC / 光模块管理；
- BMC / IPMI / Redfish 带外网络；
- 机柜级资产标签和端口映射；
- 备件、维修通道和维护窗口。

4.2 功率预算

机柜总功率 =
 $\Sigma(\text{服务器额定功率} \times \text{峰值系数})$
+ 交换机功率
+ 存储功率
+ 管理设备功率
+ 冗余余量

必须留出：

- 10% ~ 20% 安全余量；
- PSU N+1 / N+N；
- 启动浪涌；
- 冷却失效时的降频策略；
- 供电中断时的优雅停机。

4.3 散热策略

方案	适用
风冷	低功率密度、旧代 GPU、小规模
冷板液冷	高功率 GPU、GB 系列、B 系列等
浸没式液冷	特殊场景，维护和生态要求高

液冷集群额外要求：

- CDU 监控和告警；
- 流量、压力、温度、露点；
- 漏液检测和快速切断；
- 快接头标准和维护规范；
- 水冷头、软管和备件周期。

5. 网络设计

5.1 四张网络

网络	作用	要求
带外管理网	BMC / Redfish / 开关机 / 固件	与业务网隔离，禁公网
业务管理网	SSH、K8s Control Plane、监控	高可用、可审计
计算网	GPU 间 All-Reduce / All-to-All / KV 传输	高带宽、低延迟、RDMA
存储网	数据集、Checkpoint、模型读取	高吞吐、稳定尾延迟
可选：前端网	用户 API / Ingress	通用 TCP/TLS

建议物理隔离或至少逻辑隔离，BMC 网络不能与业务网络混用。

5.2 InfiniBand vs RoCE / Ethernet

方案	优点	挑战
InfiniBand	成熟低延迟、生态强、拥塞控制成熟	成本、交换机和运维体系
RoCEv2	Ethernet 生态、统一网络	PFC / ECN / QoS / 拥塞调优复杂
NVIDIA Spectrum-X	面向 AI Ethernet, 提供更完整方案	受供应商和平台约束

无论选哪种：

- 使用 RDMA 和 GPUDirect RDMA;
- 设计非阻塞或低收敛比;
- 采用 spine-leaf 与 rail-optimized;
- 监控 PFC、ECN、丢包、重传、端口错误;
- 做多轨、故障域和拓扑感知调度。

5.3 互联验证

<pre>nvidia-smi topo -m nvidia-smi nvlink -s ibstat ibv_devinfo rdma link</pre>

必须分别验证：

- GPU 到 GPU：NVLink / PCIe;
- 节点到节点：RDMA 带宽和延迟;
- GPU 到 NIC：GPUDirect RDMA;
- 跨 leaf / spine 的尾延迟;
- 多轨并发和网络拥塞。

6. 存储设计

6.1 数据分层

层	内容	介质
热数据	当前模型、KV Cache Offload、临时数据	本地 NVMe / 高性能并行 FS
温数据	常用模型、数据集、Checkpoint	并行文件系统 / NAS
冷数据	历史模型、日志、归档	对象存储
元数据	模型版本、数据集版本、血缘	数据库 / Catalog

6.2 文件系统与对象存储

- 训练 / Checkpoint：优先支持 POSIX、高吞吐、可扩展元数据的并行文件系统;
- 模型仓库：对象存储 + 版本 + 校验和;
- 日志 / Trace：对象存储或日志平台;
- 向量数据：pgvector / Milvus / 专用向量库;
- 缓存：LMCache / Redis / 本地 NVMe。

6.3 存储验收

必须测：

- 顺序读 / 写;
- 随机读 / 写;

- 多客户端并发；
- 大文件和小文件；
- Checkpoint 写突发；
- 故障降级和恢复；
- GPU Direct Storage（如使用）。

工具： fio、mdtest、ior、厂商基准和业务真实 workload。

7. 软件栈与节点初始化

7.1 基线软件

- OS：企业 Linux，固定内核和驱动版本；
- NVIDIA Driver、CUDA、cuDNN、NCCL；
- NVIDIA Container Toolkit；
- GPU Operator；
- Network Operator / RDMA 驱动；
- DCGM / DCGM Exporter；
- Node Feature Discovery；
- Container Runtime；
- Prometheus / Grafana / Loki 或 OpenTelemetry；
- Slurm 或 Kubernetes。

7.2 自动化原则

- 所有节点由代码和镜像创建；
- BIOS、BMC、驱动和固件版本进入基线；
- 节点有唯一资产 ID、机柜、U 位、交换机和端口映射；
- 用 Ansible / Terraform / GitOps 管理配置；
- 升级必须支持分批和回滚。

7.3 裸机配置流程

BMC 上线
→ Redfish 配置 RAID / BIOS / Boot
→ PXE / MAAS / Tinkerbell 安装 OS
→ 驱动和 CUDA 基线
→ RDMA / NCCL / 存储验证
→ 加入 K8s / Slurm
→ 监控与安全 Agent
→ Burn-in 和节点验收
→ 纳入调度池

8. Slurm 还是 Kubernetes

维度	Slurm	Kubernetes
典型场景	HPC、训练、批处理	推理服务、平台、微服务
作业排队	成熟	需 Kueue / Volcano 等
服务弹性	较弱	强
多租户	账号 / 分区 / QOS	Namespace / RBAC / Kueue
GPU 拓扑	原生强	需 DRA / device plugin / topology-aware
推理发布	需额外服务层	Deployment / KServe / Ray

大量企业采用：

Slurm：训练、HPC、离线批处理
Kubernetes：在线推理、Agent、RAG、平台服务

不要为了“统一”强行用一种工具覆盖所有 workload。

9. 模型部署路径

9.1 引擎选择

场景	推荐
通用推理 / 易用	vLLM
复杂控制流 / Agent / RAG	SGLang
NVIDIA 极致优化	TensorRT-LLM
企业容器服务	NVIDIA NIM
分布式、Prefill/Decode、KV 路由	NVIDIA Dynamo / llm-d

9.2 部署形态

场景	形态
单模型、小规模	vLLM + Docker
多副本推理	K8s + Ray Serve / KServe
多租户	Kueue + 网关 + 配额
多机大模型	TP + PP + RDMA
长上下文	Prefix Cache + KV Offload / LMCache
高吞吐在线服务	Prefill / Decode 分离 + KV-aware routing

9.3 发布要求

- 模型、Tokenizer、Adapter 和量化版本一起锁定；
- 镜像固定 digest；
- 有 Canary、回滚、健康检查和模型预热；
- 有最大并发、队列、超时和熔断；
- 有租户配额和成本统计。

10. 调优路径：从物理层到应用层

10.1 物理 / 集群层

- 检查 GPU 拓扑和 NUMA；
- 使用正确的 NUMA / CPU / NIC 亲和；
- 使用 NVLink / GPUDirect RDMA；
- 多轨网卡和 rail-optimized 调度；
- MIG / MPS 隔离；
- 避免同一节点混跑不同通信模式任务；
- 检查 HBM 温度、功耗和降频；
- 检查存储吞吐和 Checkpoint 争用。

10.2 模型 / 引擎层

参数	目标
gpu-memory-utilization	平衡权重、KV Cache 和并发
max-model-len	控制 KV Cache 上限
max-num-seqs	控制批大小和延迟
max-num-batched-tokens	控制 Prefill 和 Batch 平衡
Prefix Cache	提高共享前缀命中
Chunked Prefill	降低长 Prompt 阻塞
Attention Backend	匹配硬件和模型
Quantization	AWQ / GPTQ / FP8 / FP4 等
Speculative Decoding	降低 Decode 延迟
CUDA Graph	降低 Kernel 启动开销
TP / PP / EP	匹配模型和互联
LoRA / Multi-LoRA	多租户适配器复用

10.3 服务层

- 并发和队列长度;
- 超时、重试、熔断和降级;
- Prefix / KV-aware routing;
- 多副本和自动扩缩容;
- 请求分级和优先级;
- 模型预热和冷启动;
- 按业务设置 SLO。

10.4 调优闭环

建立基线 → 只改一个变量 → 压测 → 看 TTFT / TPOT / 吞吐 / 显存 / 网络 / 成本 → 保存配置和结果 → 回滚或固化

11. 4 周物理集群专项（可接在 32 周路线后）

第 1 周：选型与设计

产物：

- GPU / CPU / NIC / 交换机 / 存储 BOM;
- 机柜图、端口表和 IP 规划;
- 网络拓扑和故障域;
- 功率、制冷和承重计算;
- SLO 和验收标准。

第 2 周：裸机与基础软件

产物：

- BMC / BIOS / 固件基线;
- OS 镜像和自动化安装;

- Driver / CUDA / NCCL / Container Toolkit;
- GPU、NVLink、RDMA、存储基础验证。

第 3 周：调度与模型服务

产物：

- K8s + GPU Operator 或 Slurm;
- 网络 / 存储 / 监控接入;
- vLLM 多副本部署;
- 网关、配额、限流、回滚。

第 4 周：验收、调优与故障演练

产物：

- Burn-in 报告;
- NCCL / RDMA / 存储 / 推理基准;
- 调优报告;
- 故障演练和 Runbook;
- 资产、版本和容量文档。

12. 物理集群交付清单

- [] 机柜、电力、制冷、网络和存储设计通过评审。
- [] 每台机器有资产 ID、机柜、U 位、BMC、端口和保修信息。
- [] OS、驱动、CUDA、NCCL、固件版本统一且有基线。
- [] BMC 网络隔离，默认密码已更换，RBAC 和审计开启。
- [] GPU、NVLink、RDMA、NCCL、存储测试通过。
- [] GPU 节点已加入调度系统并打上 topology label。
- [] 监控覆盖 GPU、网络、存储、节点、调度和服务。
- [] 有队列、配额、优先级和抢占策略。
- [] 有模型部署、Canary、回滚和模型预热流程。
- [] 有故障检测、隔离、Drain、维修、返修和恢复流程。
- [] 有功率、利用率、Token 成本和容量报表。
- [] 有完整的 Runbook、资产库和验收报告。

第六部分：GPU 集群监控、调试、故障处理与验收

来源文件：08_GPU集群监控调试故障处理与验收.md

GPU 集群监控、调试、故障处理与验收

本文覆盖生产 GPU 集群的监控、基线测试、告警、故障定位、隔离、Drain、维修、返修和恢复。命令必须在测试环境验证后再进入生产；涉及重启、清空、重置或下电的操作必须遵守企业变更流程。

1. 运维闭环

Build
→ Baseline & Burn-in
→ Monitor
→ Detect
→ Classify
→ Isolate / Drain
→ Remediate
→ Validate
→ Return to Pool
→ Postmortem

没有验证和复盘的维修，不算闭环。

2. 监控体系

2.1 监控分层

层	监控对象	工具
物理	电力、温度、风扇、液冷、漏液、门禁	DCIM / Redfish / OpenBMC
节点	CPU、内存、磁盘、NIC、BMC	Node Exporter
GPU	利用率、显存、ECC、XID、温度、功耗、NVLink	DCGM / DCGM Exporter
网络	端口、带宽、PFC、ECN、丢包、RDMA 错误	交换机 Telemetry / NIC Exporter
存储	带宽、IOPS、延迟、容量、元数据	存储平台 / Node Exporter
调度	队列、等待、配额、抢占、利用率	Slurm / Kueue / Ray
推理	TTFT、TPOT、QPS、KV Cache、错误	vLLM / Gateway / Prometheus
应用	任务成功、工具调用、人工接管、成本	OTel GenAI / Langfuse / MLflow

2.2 GPU 关键指标

指标	说明	常见告警
GPU Utilization	计算利用率	长时间 0% 或 100% 都可能异常
Framebuffer Memory	显存使用	接近上限、突增、持续不释放
ECC Corrected	可纠正 ECC	持续增长需要跟踪
ECC Uncorrected	不可纠正 ECC	立即隔离和维修
XID	NVIDIA 驱动错误码	按 XID 分类处理
Temperature	GPU / HBM 温度	过热、降频、风扇故障
Power	功耗和上限	异常降频、供电问题
NVLink	链路状态和错误计数	链路降速、CRC、掉链
PCIe	带宽和错误	Replay、降速、掉设备
Clocks Throttle	降频原因	温度、功耗、电源或外部限制
GPU Reset / Fall off Bus	复位 / 掉卡	节点隔离和重启验证

2.3 网络关键指标

- 端口 Up / Down;
- 带宽和利用率;
- PFC Pause 和 Xon / Xoff;
- ECN 标记;
- 丢包、重传、CRC;
- RDMA retransmit、out-of-sequence;
- 交换机 Buffer 使用;
- 跨 leaf / spine 尾延迟;
- RoCE / InfiniBand 链路错误。

2.4 推理关键指标

- TTFT P50 / P95 / P99;
- TPOT / ITL;
- Output token/s;
- Request throughput;
- Queue time;
- KV Cache 使用率;
- Prefix Cache Hit Rate;
- GPU / CPU / Network 利用率;
- 错误率、超时、重试;
- 每百万 Token 成本。

2.5 告警级别

级别	示例	动作
P0	不可纠正 ECC、GPU 掉卡、整集群不可用	立即隔离，通知 On-call
P1	NCCL 超时、链路降速、节点频繁重启	30 分钟内响应
P2	温度偏高、ECC 可纠正增长、队列积压	当日处理
P3	容量趋势、固件过期、日志异常	计划处理

3. 基线检查命令

3.1 节点和 GPU 总览

```
hostnamectl
uname -a
lscpu
free -h
lsblk
ip link
nvidia-smi
nvidia-smi topo -m
nvidia-smi -q
```

3.2 详细健康信息

```
nvidia-smi -q -d ECC
nvidia-smi -q -d TEMPERATURE
nvidia-smi -q -d POWER
nvidia-smi -q -d PERFORMANCE
nvidia-smi -q -d XID
nvidia-smi nvlink -s
nvidia-smi nvlink -e
```

3.3 内核和 NVIDIA 日志

```
dmesg -T | grep -Ei 'nvrml|xid|ecc|nvlink|fallen|pcie|aER|mce'
journalctl -k --since '24 hours ago' | grep -Ei 'nvrml|xid|ecc|nvlink|pcie'
```

3.4 DCGM 诊断

```
dcgmi discovery -l
dcgmi diag -r 1
dcgmi diag -r 2
dcgmi diag -r 3
```

等级越高，耗时和影响力越大。生产诊断必须安排窗口。

3.5 NCCL 和 RDMA

```
ibstat
ibv_devinfo
rdma link
nvidia-smi topo -m
```

NCCL 测试示意：

```
NCCL_DEBUG=INFO \
NCCL_DEBUG_SUBSYS=INIT,NET,GRAPH \
./all_reduce_perf -b 8 -e 8G -f 2 -g 8
```

常用环境变量：

```
NCCL_DEBUG=INFO
NCCL_DEBUG_SUBSYS=INIT,NET,GRAPH
NCCL_SOCKET_IFNAME=<management-interface>
NCCL_IB_HCA=<rdma-device-list>
NCCL_IB_GID_INDEX=<gid-index>
```

不要在无法解释参数含义的情况下直接复制到生产。

4. 验收与 Burn-in 测试

4.1 节点验收顺序

1. BMC / Redfish 和资产信息；
2. BIOS / 固件 / 驱动 / OS 基线；
3. CPU、内存、磁盘；
4. GPU 识别和 ECC；
5. GPU 压力测试；
6. NVLink / PCIe；
7. RDMA / NCCL；
8. 存储；
9. 长时间稳定性；

10. 加入调度前的健康证明。

4.2 GPU Burn-in

gpu-burn 用于发现显存和计算稳定性问题：

```
git clone https://github.com/wilicc/gpu-burn.git
cd gpu-burn
make
./gpu_burn 1800
```

同时监控：

- 温度；
- 功耗；
- ECC；
- XID；
- 降频原因；
- 系统日志。

4.3 NVLink / PCIe

```
nvidia-smi topo -m
nvidia-smi nvlink -s
nvidia-smi nvlink -e
./nvbandwidth
```

检查：

- 所有链路 Up；
- 预期带宽；
- CRC / Replay 增长；
- P2P 是否正常；
- 跨 NUMA 是否有意外路径。

4.4 NCCL

必测：

```
all_reduce
all_gather
reduce_scatter
broadcast
all_to_all
```

记录：

- 小消息延迟；
- 大消息带宽；
- 多节点扩展效率；
- 是否存在某一条 rail / NIC 明显变慢。

4.5 RDMA

```
ib_write_bw
ib_read_bw
ib_send_bw
```

验证：

- 单轨带宽；

- 多轨并发；
- 跨 leaf / spine；
- 丢包和 PFC；
- GPU Direct RDMA。

4.6 存储

```
fio --name=read --rw=read --bs=1M --size=10G --numjobs=16 --group_reporting
fio --name=randread --rw=randread --bs=4k --iodepth=64 --numjobs=16 --group_reporting
```

测试：

- 顺序 / 随机；
- 大文件 / 小文件；
- 多客户端并发；
- Checkpoint 突发；
- 故障降级。

4.7 推理

```
vlm bench serve \
--base-url http://gateway/v1 \
--model qwen2.5-7b \
--dataset-name sharegpt \
--num-prompts 500 \
--max-concurrency 32
```

必须跑：

- 单副本；
- 多副本；
- Prefix Cache 开 / 关；
- 不同上下文长度；
- 不同输出长度；
- 故障副本。

5. 故障分类与处理流程

5.1 故障分类

类别	例子
硬件	GPU、HBM、NVLink、NIC、交换机、电源、风扇
固件 / 驱动	版本不兼容、驱动崩溃、NVLink 降速
网络	RDMA 断链、PFC 风暴、丢包、NCCL 超时
存储	延迟升高、IO 错误、元数据瓶颈
调度	队列饥饿、配额错误、拓扑放置不佳
推理引擎	OOM、引擎崩溃、KV Cache 不足、模型加载失败
应用	Prompt 过长、无限循环、工具超时、越权
安全	凭据泄露、越权、供应链漏洞、数据外泄

5.2 标准处理流程

1. 确认影响范围
2. 保存现场：日志、指标、XID、NCCL、dmesg、Pod Events
3. 判断是单卡、单节点、单轨、单交换机还是全局
4. 隔离或 Drain
5. 执行 Runbook
6. 修复或返修
7. 重新验证
8. 回池并观察
9. 复盘和补充监控

6. 常见故障 Runbook

6.1 XID 错误

现象： dmesg、NVIDIA 日志或 DCGM 出现 XID。

处理：

1. 查询 NVIDIA XID Errors 文档；
2. 记录 XID 编号、GPU、节点、时间和 workload；
3. 判断是否可恢复、需要复位、需要隔离或 RMA；
4. 有不可纠正错误时，不要直接把节点重新加入池；
5. 执行 dcgmi diag 和压力测试；
6. 多次复现则换卡或返修。

6.2 ECC 错误

可纠正 ECC：

- 记录增长速率；
- 如果短期激增，降低流量并加深诊断；
- 观察是否伴随 XID、温度或性能下降。

不可纠正 ECC：

- 立即隔离 GPU / 节点；
- 停止新任务；
- 保存现场；
- 诊断或返修；
- 未通过验证前不得回池。

6.3 GPU 掉卡 / 掉总线

现象： nvidia-smi 找不到卡、PCIe 错误、驱动日志异常。

处理：

1. 通过 BMC 检查硬件日志；
2. 检查 GPU、PCIe、电源、散热和插槽；
3. 判断是否可热重置；
4. 不能恢复则重启 / 更换；
5. 重新执行 Burn-in 和 NCCL。

6.4 NVLink 异常

```
nvidia-smi nvlink -s  
nvidia-smi nvlink -e  
nvidia-smi topo -m
```

处理:

- 记录哪条 GPU-GPU 链路错误;
- 检查线缆、连接器和背板;
- 重新初始化或重启节点;
- 未恢复则隔离或维修;
- 验证 TP / NCCL 性能恢复。

6.5 NCCL 超时 / 挂起

排查顺序:

1. 哪两个 rank 之间超时?
2. 是否单节点、单机架或单交换机范围?
3. 是否一张卡温度 / ECC / XID 异常?
4. 是否 RDMA 链路、GID、路由或 MTU 问题?
5. 是否 NUMA / NIC 选错?
6. 是否某个 rank 因 OOM 或进程退出?

常用做法:

```
NCCL_DEBUG=INFO  
NCCL_DEBUG_SUBSYS=INIT,NET,GRAPH
```

不要盲目加上 `NCCL_IB_DISABLE=1`, 那会把问题掩盖成性能退化。

6.6 RDMA / RoCE 故障

检查:

```
ibstat  
ibv_devinfo  
rdma link  
ethtool -S <interface>
```

重点:

- 链路速率和状态;
- GID / MTU;
- PFC、ECN、丢包;
- RoCE QoS;
- 交换机端口错误;
- 多轨是否偏流。

6.7 温度 / 降频

检查:

- 进出风温度;
- 风扇转速;
- GPU / HBM 温度;
- 功率上限;

- 降频原因；
- 液冷流量和压力。

处理：

- 提升冷却；
- 检查风道和滤网；
- 调整功率上限；
- 隔离异常节点；
- 记录对 TTFT / 吞吐的影响。

6.8 推理 OOM

按顺序检查：

1. 模型权重显存；
2. `max-model-len`；
3. 并发和 KV Cache；
4. `gpu-memory-utilization`；
5. 是否有其他进程占用；
6. 是否发生碎片；
7. 是否需要量化、TP 或更多副本。

不要只通过重启服务“解决”OOM。

6.9 vLLM 引擎异常

检查：

- 启动日志和模型加载；
- 端口和健康检查；
- GPU 可见性；
- KV Cache 初始化；
- 是否有 NCCL / CUDA 错误；
- 网关是否把请求发到故障副本；
- 副本能否被自动摘除。

6.10 队列积压和性能劣化

区分：

- 请求量增加；
- 模型变慢；
- GPU 降频；
- KV Cache 命中率下降；
- 网络 / 存储变慢；
- 某个租户占用过多；
- 副本数不足；
- 路由不均。

不能只看平均延迟，必须看 P95 / P99、队列时间和每副本指标。

7. 节点 Drain、维修和回池

Kubernetes

```
kubect! cordon <node>
kubect! drain <node> --ignore-daemonsets --delete-emptydir-data
# 执行维修
kubect! uncordon <node>
```

Slurm

```
scontrol update NodeName=<node> State=DRAIN Reason="gpu maintenance"
scontrol show node <node>
scontrol update NodeName=<node> State=RESUME
```

回池前必须：

- 硬件诊断通过；
- Burn-in 通过；
- NCCL / RDMA 通过；
- 推理服务健康；
- 监控正常；
- 变更记录完成。

8. 监控看板建议

集群总览

- 节点总数、GPU 总数、可用 GPU；
- 分配率、利用率、排队；
- 总功率、温度、降频节点；
- 网络健康；
- 存储容量和延迟。

GPU 节点

- GPU 利用率、显存、温度、功耗；
- ECC、XID、NVLink、PCIe；
- 进程、Pod / Job、CUDA 版本；
- 降频原因；
- 最近事件。

推理服务

- QPS、TTFT、TPOT、错误率；
- 队列、KV Cache、Prefix Hit；
- 副本数、重启、路由分布；
- Token 成本和租户使用。

网络 / 存储

- 端口速率、错误、PFC / ECN；
- RDMA 重传；
- 存储带宽、IOPS、延迟；

- Checkpoint 和数据集热点。

9. 验收报告模板

```
# GPU Cluster Acceptance Report

## Cluster
- Site / rack / nodes / GPUs:
- Firmware / driver / CUDA / NCCL:
- Fabric / storage / scheduler:

## Node burn-in
- Duration:
- Temperature / power / clock:
- ECC / XID / dmesg:

## Interconnect
- NVLink / PCIe:
- RDMA / IB / RoCE:
- NCCL small / large message:
- Multi-rail / cross-spine:

## Storage
- Sequential / random:
- Multi-client:
- Checkpoint burst:

## Inference
- TTFT / TPOT / throughput:
- Concurrency:
- Failure recovery:
- Cost per million tokens:

## Fault drills
- Scenario:
- Detection:
- Isolation:
- Recovery:
- Data loss:
- Follow-up:

## Result
- Pass / Conditional pass / Fail:
- Exceptions:
- Owner / deadline:
```

10. 最终检查表

- ☐ 节点、GPU、网卡、交换机、存储都有资产和端口映射。
- ☐ 监控覆盖物理、节点、GPU、网络、存储、调度和推理。
- ☐ 有 P0 ~ P3 告警和明确 Runbook。
- ☐ Burn-in、NVLink、NCCL、RDMA、存储和推理验收全部通过。
- ☐ 节点维修支持 Drain、隔离、返修和回池验证。
- ☐ 不可纠正 ECC、XID 和掉卡有明确处理策略。
- ☐ 推理服务的 OOM、超时、故障副本和队列积压有演练记录。
- ☐ 能计算 GPU 利用率、Token 成本和容量余量。
- ☐ 有版本基线、固件清单、备件计划和维护窗口。
- ☐ 有任何故障都能回溯到时间、节点、GPU、进程和变更记录。

第七部分：企业落地常见坑与注意事项

来源文件：10_企业落地常见坑与注意事项.md

企业落地常见坑与注意事项

目标：不是告诉学习者“功能怎么做”，而是提前标出企业 PoC、试点和规模化阶段最容易翻车的点。每个模块都按“典型错误 -> 正确做法 -> 上线前检查”组织。

1. 通用原则

企业 AI 项目失败通常不是因为模型不够强，而是因为下面几类问题：

- 1. 问题没有业务价值，只是为了用 AI；
- 2. 只有 Demo，没有异常、权限、评估、回滚；
- 3. 没有数据边界和安全评审；
- 4. 只关注模型，不关注 Harness、Infra 和运营；
- 5. 没有成本、容量和 SLO；
- 6. 没有真实用户参与，指标是技术指标而不是业务指标；
- 7. 供应商、模型和工具版本不可控；
- 8. 缺少故障演练和人工接管。

2. 分模块避坑表

2.1 Python、FastAPI 与工程基础

容易犯的问题	正确做法	上线前检查
直接在系统 Python 安装依赖	每个项目使用虚拟环境或容器	uv.lock / requirements 可复现
依赖不锁版本	锁定 Python、库和镜像版本	重建环境能通过测试
把 API Key 写在代码或 Notebook	使用 Secret Manager 或环境变量注入	仓库扫描无凭据
接口没有超时和重试上限	每个外部调用设置超时、重试和熔断	模拟下游超时
只返回 200，错误码混乱	统一错误 Schema 和 trace_id	客户端能区分可重试和不可重试错误
日志打印敏感数据	日志脱敏并保留审计字段	PII 扫描通过

2.2 Prompt 与 Context Engineering

容易犯的问题	正确做法	上线前检查
把 Prompt 当代码硬编码在业务中	Prompt 版本化、评测、灰度	能看到线上 Prompt 版本
一味增加上下文长度	做检索、摘要、压缩和去重	Token 和 TTFT 有上限
把外部文档当成可信指令	区分系统指令、用户指令和不可信数据	Prompt Injection 用例通过
只测几个成功样例	建立 Golden Set、边界集和对抗集	回归门禁通过
没有结构化输出约束	使用 JSON Schema / Pydantic	输出可解析率达标
忽略 Prompt 对成本的影响	记录输入输出 Token 和单任务成本	成本看板可用

2.3 Harness、Agent Loop 与状态

容易犯的问题	正确做法	上线前检查
Agent 可以无限循环	限制步骤、时间、Token、成本和工具次数	能触发停止条件
只存在内存中的状态	Run State 持久化到数据库	进程重启后可恢复
没有取消和重试语义	定义 cancel、retry、resume	重复执行不会重复写入
把模型最终回答直接当成功	加入验证器和验收条件	失败不能标记成功
多 Agent 没有边界	明确子 Agent 职责、上下文和预算	不存在无限委托
没有人工接管	设计人工接管和降级路径	连续失败能转人工

2.4 Memory 与 Context 压缩

容易犯的问题	正确做法	上线前检查
把所有历史都塞给模型	分层 Memory 和上下文编译	Token 有预算
记忆没有租户隔离	tenant_id + user_id 强隔离	跨租户测试通过
记忆永不删除	支持 TTL、删除、纠错和来源	用户删除流程可用
把模型总结当事实	保留来源、时间和置信度	记忆来源可追溯
工具结果过长	结构化、截断、提炼和引用	Context 不爆
记忆污染	写入前做质量、安全和敏感检查	有记忆审核策略

2.5 RAG、Embedding 与向量数据库

容易犯的问题	正确做法	上线前检查
只调 Chunk Size	按文档结构、标题、表格、父子块设计	分块评测有对比
只用向量检索	混合检索 + Rerank	Recall@K 和 MRR 达标
不处理 OCR 和表格	解析、表格序列化、版面恢复	抽样人工可读
知识库权限穿透	检索前做权限过滤	越权测试通过
知识库无版本	文档版本、索引版本、更新时间和来源	能回滚索引
引用是假的	生成后验证引用片段真实存在	引用点击可核验
RAG 评价只看答案相似度	同时看检索、忠实度、相关性、拒答	有完整评测报告
把长上下文当作 RAG 替代	先比较成本、延迟、更新频率和权限	有选型证据

2.6 Function Calling、MCP 与工具

容易犯的问题	正确做法	上线前检查
工具太多，模型无法选择	按任务分组、命名空间和最小工具集	工具选择准确率达标
工具 Schema 含糊	明确参数类型、枚举、示例和错误码	自动生成 Schema 测试
工具失败无错误边界	返回稳定错误码，不泄漏堆栈	客户端可处理
写操作无幂等	强制幂等键和去重	重试不重复写
工具权限过大	最小权限、scope 和审批	越权测试失败
MCP Server 直接暴露内网	网络隔离、认证、审计和限流	安全评审通过
工具输出不可信	工具结果当数据处理，不作指令	注入测试通过
工具版本不锁	ToolSpec 版本和兼容矩阵	版本可回滚

2.7 Sandbox、审批与 Human-in-the-Loop

容易犯的问题	正确做法	上线前检查
代码直接在宿主执行	容器沙箱、非 root、限制网络 and 文件	逃逸测试通过
所有操作都要审批	只对高风险写操作审批	审批负担可接受
审批没有参数快照	保存参数、影响范围、审批人和理由	审计可复现
审批超时状态不清楚	定义超时、拒绝、重试和取消	状态机测试通过
模型能绕过权限	权限在 Harness 外部强制执行	Prompt 注入无法越权
审批通过后无条件重放	校验幂等和参数未变化	重放安全

2.8 Verification、评测与上线门禁

容易犯的问题	正确做法	上线前检查
只看主观 Demo	建 Golden Set、失败集和对抗集	评测可重复
用同一个模型给自己打分	多模型评审 + 人工抽样 + 业务规则	偏差可接受
只测最终答案	追踪工具、检索、引用和中间步骤	Trace 完整
只看平均值	看 P95/P99、长尾和失败类型	最差用例可解释
模型/Prompt 变更不看回归	CI 自动跑评测	回归不过不允许发布
业务指标缺失	任务成功、接管率、节省时间、成本	有业务验收人

2.9 AI PDE 产品工程

容易犯的问题	正确做法	上线前检查
把“能生成”当用户价值	先做用户研究、基线和指标	有真实用户签字
只设计成功页面	设计等待、失败、重试、低置信度、审批	状态全覆盖
不显示来源和置信度	引用、证据、时间戳、适用范围	用户能核查
生成慢但无反馈	流式、进度、可停止和可恢复	真实延迟测试
没有人工接管	明确何时转人工、如何带上下文	接管流程可用
A/B 只看点击率	看任务完成、错误、成本和留存	业务收益清晰
没有回滚	Feature Flag、版本和降级方案	一键关闭可用

2.10 vLLM 与单机推理

容易犯的问题	正确做法	上线前检查
直接复制旧命令	按当前 vLLM 版本和模型要求启动	版本矩阵确认
gpu-memory-utilization 盲目拉满	平衡权重、KV Cache 和并发	OOM 测试通过
max-model-len 设置过大	按业务上下文和显存设置	并发压测通过
不预热模型	启动/发布前做预热请求	冷启动指标可接受
没有限流和队列	网关限流、超时、熔断	过载测试通过
只测单请求吞吐	测并发、长上下文和多轮对话	有 P95/P99
量化不测质量	质量、显存、吞吐同时比较	业务样例通过
模型和 Tokenizer 不匹配	锁定模型、Tokenizer、Adapter 版本	加载校验通过

2.11 Kubernetes、GPU 调度与推理平台

容易犯的问题	正确做法	上线前检查
驱动、CUDA、GPU Operator 版本不匹配	使用供应商兼容矩阵	节点验收通过
不设 taint/toleration	GPU 节点专用或按需隔离	普通 Pod 不占 GPU
只看 GPU 数量，不看拓扑	NVLink / NUMA / NIC 亲和调度	拓扑测试通过
GPU 碎片化	资源池、配额、拓扑感知调度	利用率达标
HPA 用错指标	用队列、并发、延迟或自定义指标	扩容有效
缩容到 0 导致冷启动	评估冷启动和保留最小副本	SLO 满足
Operator 版本升级无窗口	分批升级和回滚	升级演练完成
请求无队列和背压	网关队列、超时和优先级	过载不雪崩

2.12 分布式推理 TP / PP / DP

容易犯的问题	正确做法	上线前检查
盲目追求 TP 数量	先看模型、节点和网络	扩展效率有收益
跨慢网络做 TP	TP 尽量在 NVLink / 高速互联域	NCCL 性能达标
PP 配置不适配负载	调整微批和流水线并行	气泡可接受
NCCL 环境变量复制错误	明确 HCA、GID、MTU 和拓扑	nccl-tests 通过
跨 AZ 部署 TP	不跨高延迟域做高频通信	网络延迟确认
无故障重调度	节点故障时重新分配和恢复	故障演练通过
只看单次 Benchmark	多轮、长时、混合负载测试	稳定性达标

2.13 观测、监控与故障定位

容易犯的问题	正确做法	上线前检查
只看平均延迟	P50/P95/P99、队列、超时和错误	告警有阈值
只监控 GPU	节点、网络、存储、调度、应用全链路	指标覆盖完整
日志含 PII 和密钥	全链路脱敏和审计	日志扫描通过
Trace 没有业务 ID	trace_id、run_id、tenant_id 贯穿	一次投诉可追到根因
指标高基数失控	标签设计和采样	监控成本可控
告警无 Runbook	告警关联责任人和处理步骤	故障演练验证
只修服务不复盘	复盘、根因和预防项	问题不再重复

2.14 安全、合规与数据治理

容易犯的问题	正确做法	上线前检查
未做数据分类	数据分级、访问控制和保留策略	数据地图完成
Prompt Injection 无防护	不可信数据处理、工具 allowlist、人工审批	红队通过
工具权限过大	最小权限和双人审批	越权测试通过
日志和 Trace 泄露	脱敏、加密、访问审计	合规评审通过
供应链未锁版本	镜像 digest、SBOM、漏洞扫描	供应链审计通过
模型输出直接执行	校验、沙箱和人工确认	无直接执行风险
没有审计追踪	记录输入、工具、审批、版本和结果	可回放
无删除和退出机制	数据删除、模型下线、供应商退出	退出演练完成

2.15 GPU 物理集群

容易犯的问题	正确做法	上线前检查
先买卡再规划	先按 workload、SLO 和机柜能力设计	BOM 与需求一致
只算 GPU 功率	计算整柜、网络、存储、冷却和冗余	功率余量足够
BMC 接入业务网	带外网隔离、RBAC 和审计	网络隔离验证
没做 Burn-in	GPU、NVLink、NCCL、RDMA、存储全测	验收报告签字
固件和驱动不统一	版本基线和资产台账	节点一致性达标
存储成为瓶颈	Checkpoint/数据模型实测	存储尾延迟达标
备件和维修流程缺失	备件池、返修和替换流程	故障恢复时间达标
交付只看能开机	看稳定性、性能和可运维性	运行 72 小时观察

2.16 国产 GPU 集群

容易犯的问题	正确做法	上线前检查
假设 CUDA 代码可直接运行	逐框架、逐算子验证迁移	模型/算子清单完成
只看芯片峰值算力	看实测训练/推理、显存、互联和框架支持	业务 workload 基准
忽略软件生态	核对驱动、编译器、通信库、容器和 K8s	完整软件栈验收
混合多种国产卡但无抽象层	统一模型服务 API 和调度层	可替换性验证
量化/算子不支持	提前做模型适配和 fallback	质量与性能达标
供应商支持不足	明确 SLA、补丁和长期维护	支持协议落实
单一供应商锁定	双供应商、双版本、双路径	迁移演练完成
直接承诺线性扩展	用 HCCL/RCCL/集合通信实测拓扑	扩展效率真实

2.17 千卡集群规划与实施

容易犯的问题	正确做法	上线前检查
按"GPU 数量"估容量	用有效算力、利用率、故障率和 SLO 估算	容量模型可解释
忽略故障概率	千卡规模故障是常态	自愈和重调度
跨 Spine 做高频通信	rail-optimized、拓扑感知	网络性能达标
Checkpoint 同时写爆存储	错峰、分层和缓存	训练不中断
调度只看空闲 GPU	看队列、配额、优先级和抢占	公平性达标
监控数据过多但无行动	关键告警 + Runbook	噪声可控
只有一套模型服务	多副本、降级和故障切换	服务不中断
成本无分摊	租户、模型、业务和 Token 维度	账单可追踪

2.18 公有云 GPU

容易犯的问题	正确做法	上线前检查
只看 GPU 型号	同时看显存、互联、CPU、内存、网络和存储	实例基准测试
云实例没有 NVLink / RDMA	购买前确认拓扑和通信能力	EFA/IB/RoCE 测试
忽略数据出站和存储费用	计算总拥有成本	TCO 可解释
Spot 抢占导致任务丢失	Checkpoint、重试和资源池回退	抢占演练
配额和区域容量不足	提前申请配额和预留容量	扩容测试
自动关机策略缺失	预算、标签和定时关机	空闲资源归零
安全组和 IAM 过宽	最小权限、私网和审计	安全基线通过
直接绑定某云 SDK	通过兼容层和标准接口	迁移测试完成

3. 企业上线红线

以下任意一项未通过，不建议进入真实业务流量：

- 没有数据分类和权限模型；
- 没有 Prompt Injection 和越权测试；
- 没有模型/Prompt/工具版本记录；
- 没有 Trace 和审计；
- 没有评测集和回归门禁；
- 没有超时、限流、熔断和降级；
- 没有人工接管和一键关闭；
- 没有成本预算和容量上限；
- 没有故障演练和 Runbook；
- 没有真实用户验收指标。

4. P0 上线检查清单

- ☐ 谁负责模型、Harness、工具、数据和 GPU 平台？
- ☐ 数据能否出内网？是否能删除？
- ☐ 哪个工具可以写数据？谁审批？
- ☐ 模型失败时用户看到什么？
- ☐ 工具故障时是否能降级？
- ☐ 模型副本故障时是否能自动摘除？
- ☐ 评测集在哪里，谁维护？
- ☐ 发布如何回滚？
- ☐ Token / GPU / 存储成本上限是多少？
- ☐ 故障时 5 分钟内能否定位到层级？
- ☐ 审计日志保存多久？
- ☐ 国产卡、云卡、NVIDIA 卡之间如何替换？

5. 配套视频与资料

- [OWASP LLM Top 10](#)
- [NIST AI RMF](#)
- [OpenTelemetry GenAI](#)
- [promptfoo](#)
- [Ragas](#)
- [vLLM Optimization](#)
- [GPU Operator](#)
- [Kueue](#)

第八部分：国产 GPU 与千卡集群规划实施指南

来源文件：11_国产GPU与千卡集群规划实施指南.md

国产 GPU 与千卡集群规划实施指南

资料核验日期：2026-10-04。国产加速器产品、驱动、编译器、通信库和框架适配变化很快。本文给出选型、规划、实施和验收方法，具体型号、算力、互联和软件版本必须以供应商当期文档、兼容矩阵和实测结果为准。

核心原则：不要把“国产 GPU”当成一个统一产品。不同厂商的指令集、运行时、通信库、算子覆盖、容器和调度方式差异很大。国产化集群最重要的不是“能插卡”，而是模型、框架、算子、通信、调度、监控和运维能否形成完整闭环。

1. 国产 GPU 集群与通用 GPU 集群的差异

维度	通用 NVIDIA 集群	国产 GPU / NPU 集群
软件栈	CUDA、NCCL、TensorRT 等生态成熟	CANN、Neuware、DTK、MUSA、BIRENSUPA、MXMACA 等多栈并存
代码迁移	大多数框架默认支持	需要逐模型、逐算子、逐精度验证
通信库	NCCL / NVLink / InfiniBand	HCCL、MCCL、RCCL 或厂商自研集合通信
调度	GPU Operator + K8s / Slurm	厂商 Operator / Device Plugin / MindCluster 等
监控	DCGM 生态成熟	厂商自带监控 + Prometheus + 自研采集
供应链	受制于国际供应与政策	国产化、自主可控、信创适配
风险	供应和出口限制	软件生态、兼容性、性能和供应商锁定
选型重点	型号、显存、NVLink、网络	生态适配、算子覆盖、通信、迁移成本和长期支持

2. 主流国产算力生态地图

2.1 厂商与平台速查

厂商	代表硬件 / 形态	软件栈与集群能力	典型定位	注意事项
华为昇腾	Atlas 服务器、910 系列、A3 SuperPoD、CloudMatrix 等	CANN、MindSpore、MindIE、MindCluster、HCCL	大模型训练、推理、超节点	需核对 CANN / 框架 / 模型版本，使用官方迁移工具
寒武纪	思元 / MLU 系列	Neuware、BANG、MagicMind 等	训练、推理、数据中心	逐模型验证算子、精度和量化支持
海光	深算 / DCU 系列	DCU 驱动、DTK 等	信创、通用计算、AI 推理训练	重点确认 ROCm 兼容层和框架版本
摩尔线程	MTT S 系列、KUAE 智算集群	MUSA、MUSACODE、MUSA Deploy、KUAE 云原生套件	训推一体、图形 + AI、智算中心	需要验证 CUDA/MUSA 迁移、模型和算子覆盖
壁仞科技	壁砺系列 GPU	BIRENSUPA、光互连 / OCS 等	数据中心训练、推理	关注集群互联和框架适配
昆仑芯	昆仑芯 1/2/3 代、R200 / R480 等	XPU / PaddlePaddle 生态、集群方案	搜索推荐、大模型训推	确认框架、算子和调度生态
沐曦	曦思、曦云、曦彩、曦索系列	MXMACA 异构计算平台、GPU 超节点	训练、推理、科学计算	关注多卡 / 多机、显存和算子兼容
天数智芯	天垓、彤央、智铠系列	天数智算软件栈	训练、推理、行业应用	确认模型迁移、通信库和容器支持
其他	燧原、昆仑、国产 NPU 等	各自软件栈	特定行业 / 信创场景	不做“只看峰值算力”的横向比较

2.2 官方入口

- 华为昇腾：<https://www.hiascend.com/>

- 昇腾 CANN: <https://www.hiascend.com/cann>
- MindIE: <https://www.hiascend.com/developer/software/mindie>
- MindCluster: <https://www.hiascend.com/developer/software/mindcluster>
- 华为超节点: <https://www.huawei.com/en/news/2025/9/hc-superpod-innovation>
- 寒武纪: <https://www.cambricon.com/>
- 海光: <https://www.hygon.cn/>
- 摩尔线程: <https://www.mthreads.com/>
- 摩尔线程开发者: <https://developer.mthreads.com/>
- 壁仞科技: <https://www.birentech.com/>
- 昆仑芯: <https://www.kunlunxin.com/>
- 沐曦: <https://www.metax-tech.com/>
- 天数智芯: <https://www.iluvatar.com/>

3. 国产 GPU 选型方法

不能只比较“单卡峰值 TFLOPS”。应按下面的顺序选型：

3.1 先确定 workload

场景	关键要求
大模型预训练	显存容量、互联带宽、集合通信、Checkpoint、稳定性
SFT / LoRA 微调	混合精度、训练框架、算子覆盖、数据吞吐
在线推理	显存、量化、批处理、KV Cache、延迟、并发
RAG / Embedding	向量计算、批量推理、吞吐和成本
多模态	图像编码、视觉算子、显存和带宽
科学计算	FP64 / 特定算子 / 数学库，不是所有 GPU 都适合

3.2 再看硬指标

- 显存容量和 HBM 带宽；
- FP16 / BF16 / FP8 / INT8 / INT4 支持；
- 单卡到多卡互联方式；
- 节点内互联带宽和拓扑；
- 跨节点 RDMA / RoCE / 自研网络；
- CPU、内存和 PCIe 配比；
- 功耗、散热和机柜密度；
- 监控、ECC 和故障遥测能力。

3.3 最后看软件生态

- PyTorch / MindSpore / PaddlePaddle 等框架支持；
- Transformers、vLLM、SGLang、TensorRT-LLM 或厂商推理引擎适配；
- 算子覆盖：Attention、MoE、FlashAttention、量化、LoRA；
- 通信库：HCCL、MCCL、RCCL 或厂商集合通信；
- 容器和 K8s Device Plugin / Operator；
- 模型迁移工具和性能分析工具；
- 供应、驱动、固件和支持周期。

3.4 选型评分表

维度	权重建议	评分方法
目标模型支持	25%	真实模型能否加载、训练、推理
算子与框架兼容	20%	逐算子测试 + 框架版本矩阵
显存与性能	15%	真实 workload 压测，不看宣传峰值
集群互联与通信	15%	HCCL/RCCCL/MCCL 集合通信测试
容器、K8s、调度	10%	Device Plugin / Operator / 资源治理
监控与故障处理	5%	温度、ECC、掉卡、日志、诊断工具
供应商支持	5%	SLA、补丁、驻场、长期维护
迁移和退出成本	5%	是否可替换，是否有双路径

4. 千卡集群规划

以下为工程方法和公式框架，不是某个厂商的采购承诺。所有系数必须用真实 workload 实测。

4.1 先定义目标

- 是训练、微调、推理还是混合负载？
- 目标模型参数、序列长度、并发和吞吐是多少？
- 目标 SLO：训练吞吐、TTFT、TPOT、成功率、可用性？
- 计划使用什么并行策略：DP、TP、PP、EP、3D 并行？
- 主要框架和推理引擎是什么？
- 数据量和 Checkpoint 频率是多少？
- 故障率、维护窗口和备件策略是什么？

4.2 容量粗估

可用有效算力 =
GPU/NPU 数量
× 单卡有效算力
× 并行扩展效率
× 集群可用率
× 业务利用率

不能直接使用“卡数 × 峰值算力”，因为：

- 通信会损失扩展效率；
- 故障和维护会降低可用率；
- 数据加载、Checkpoint 和调度会损失利用率；
- 量化、算子和框架兼容会改变有效性能。

4.3 网络规划

千卡集群的网络通常分为：

计算网：TP / PP / EP / All-Reduce / All-to-All
存储网：训练数据、Checkpoint、日志、对象存储
管理网：调度、监控、证书、镜像仓库
带外网：BMC / Redfish / 开关机 / 固件
前端网：API / Ingress / 用户流量

必须明确：

- 节点内互联和跨节点互联；
- spine-leaf 还是 rail-optimized；

- 是否有 RDMA / RoCE / InfiniBand / 自研网络；
- 阻塞比、收敛比和网络故障域；
- 跨 spine 的延迟和带宽；
- PFC、ECN、QoS、拥塞控制和多轨负载均衡。

4.4 存储规划

存储吞吐 $\approx$
 (Checkpoint 大小 / Checkpoint 间隔)
 $\times$ 安全系数
 $+$ 数据集读取吞吐
 $+$ 模型加载和镜像吞吐

还要考虑：

- 大文件顺序吞吐；
- 小文件元数据；
- 多客户端并发；
- Checkpoint 同时写入；
- 模型权重缓存；
- 故障降级和恢复；
- 对象存储与并行文件系统的分层。

4.5 供电与散热

机柜总功率 =
 计算节点
 $+$ 交换机
 $+$ 存储
 $+$ 管理设备
 $+$ 制冷设备
 $+$ 安全余量

千卡集群必须关注：

- 单柜功率密度；
- PDU、UPS、母线和冗余；
- 启动浪涌；
- 风冷或液冷能力；
- CDU、漏液检测和水温；
- 故障降频和优雅停机。

4.6 故障域设计

千卡规模下，故障是常态，不是异常。规划时必须考虑：

- 单卡故障；
- 单节点故障；
- 单交换机 / 单 leaf / 单 spine 故障；
- 单存储节点故障；
- 单机柜供电或制冷故障；
- 网络拥塞和慢节点。

目标是：

发现 -> 隔离 -> 重调度 -> 恢复 -> 验证 -> 回池

5. 分阶段实施路线

阶段 1：8~32 卡 PoC

- 验证目标模型能加载和运行；
- 验证框架、算子、量化、推理引擎；
- 做 NVLink / HCCL / RCCL / MCCL 等通信测试；
- 建立单节点性能基线；
- 明确迁移改造点。

阶段 2：64~128 卡试点

- 多机通信、调度、存储和监控；
- 训练 / 推理混合调度；
- 配额、优先级和故障恢复；
- 业务真实 workload；
- 小规模用户试用。

阶段 3：256~512 卡平台化

- 多租户、队列、配额、抢占；
- 模型服务多副本和灰度；
- 统一模型仓库和评测；
- 监控、告警、容量和成本；
- 故障演练和备件机制。

阶段 4：千卡及以上

- 拓扑感知调度；
- rail-optimized 网络；
- Checkpoint 错峰和分层存储；
- 自动重调度和慢节点隔离；
- 硬件故障预测与预防性维护；
- 跨集群和多供应商资源池。

6. 国产软件栈迁移要点

6.1 分层解耦

业务应用
→ 统一模型 API / OpenAI 兼容协议
→ Harness / 模型网关
→ 推理引擎适配层
→ 厂商运行时与通信库
→ 国产 GPU / NPU

业务代码不能直接依赖某个厂商的 SDK。所有厂商相关代码放在适配层。

6.2 迁移检查清单

- [] 模型权重格式和 Tokenizer 兼容；
- [] Attention、MoE、RoPE、量化算子支持；
- [] 分布式训练 / 推理并行策略支持；
- [] 集合通信库支持和拓扑验证；

- [] 精度对齐：FP32 / BF16 / FP16 / FP8;
- [] 容器镜像、驱动和固件匹配;
- [] K8s Device Plugin / Operator 可用;
- [] 监控、日志、ECC、XID 类似事件可观测;
- [] 供应商支持和问题升级路径;
- [] 有回退到 NVIDIA 或另一家国产卡的能力。

7. 千卡集群验收

7.1 单节点

- 驱动、固件、OS 基线;
- 显存、温度、功耗、ECC;
- 单卡压力测试;
- 节点内互联;
- 本地存储和网络。

7.2 多节点

- 集合通信：All-Reduce、All-Gather、Reduce-Scatter、All-to-All;
- 小消息延迟、大消息带宽;
- 单轨、多轨、跨 leaf、跨 spine;
- 训练步耗时;
- 模型加载时间;
- Checkpoint 写入和恢复。

7.3 推理服务

- 模型加载和预热;
- TTFT、TPOT、吞吐、并发;
- KV Cache、Prefix Cache;
- 量化精度;
- 多副本、故障摘除和恢复;
- 单位 Token 成本。

7.4 故障演练

- 单卡故障;
- 单节点故障;
- 单交换机 / 单链路故障;
- 存储故障;
- 调度器故障;
- 供电 / 制冷告警;
- 慢节点和网络拥塞。

8. 视频与资料

国产集群

- [昇腾 CANN 入门课](#)

- [Ascend C 算子开发入门](#)
- [MindIE NPU 模型部署指南](#)
- [基于昇腾超节点的大模型预训练实践](#)
- [GPU 万卡集群概览](#)
- [寒武纪 MLU370 安装](#)
- [寒武纪软件栈基础课程](#)
- [摩尔线程 MUSA 编程课堂](#)
- [MUSA SDK 软件栈介绍](#)
- [沐曦 MXMACA 编程教程](#)

千卡 / 大规模集群

- [How to Design a GPU Cluster for AI Training](#)
- [GPU Cluster Network Design for AI Training](#)
- [Building a GPU Cluster for AI](#)
- [The Hidden Crisis Behind Large-Scale GPU Clusters](#)
- [Inside a New AI Cluster with NVIDIA B200](#)

9. 最终检查表

- [] 目标模型在国产卡上能加载、训练和推理。
- [] 关键算子和精度已对齐。
- [] 集合通信和拓扑已实测。
- [] 千卡容量、网络、存储、功率和制冷模型已评审。
- [] 调度、配额、抢占和故障恢复已演练。
- [] 监控覆盖卡、节点、网络、存储、调度和服务。
- [] 供应商兼容矩阵、SLA 和升级路径明确。
- [] 有第二供应商或 NVIDIA 回退路径。
- [] 有真实 workload 的性能、质量和成本数据。

第九部分：公有云 GPU 选型与实施要点

来源文件：12_公有云GPU选型与实施要点.md

公有云 GPU 选型与实施要点

资料核验日期：2026-10-04。云厂商实例族、GPU 型号、配额、区域可用性和计费方式变化很快，采购和上线前必须重新查看云厂商官方文档和当前区域库存。

核心原则：云 GPU 不是“按卡租赁”这么简单。同一个 GPU 型号在不同实例、区域和网络配置下，显存、CPU、内存、NVLink、RDMA、存储和计费可能完全不同。

1. 公有云 GPU 与自建 GPU 集群

维度	公有云 GPU	自建 GPU 物理集群
启动速度	分钟到小时	周 / 月级采购
运维责任	云厂商负责底层，用户负责实例和服务	用户负责机房、供电、制冷、网络和硬件
成本	按量、预留、Spot、包年包月	CapEx + 电费 + 运维
弹性	强，但受配额和库存限制	固定，扩容周期长
网络	受实例和区域限制	可自定义
数据安全	依赖云安全和合规	完全自控，但责任更大
适合	PoC、潮汐业务、训练峰谷、快速扩容	长期稳定、高利用率、合规私有化

建议：

- 学习和 PoC：优先公有云 GPU；
- 稳定高利用率：测算自建与云的成本交叉点；
- 峰值训练：云 + 预留容量 + Spot 混合；
- 核心生产：私有化 / 专有云 + 云灾备或多云。

2. 选型维度

2.1 GPU 层

- GPU 型号和代际；
- 显存容量和 HBM 带宽；
- FP16 / BF16 / FP8 / INT8 / FP4 支持；
- 单实例 GPU 数量；
- 是否支持 MIG / vGPU / 切分；
- 是否支持 NVLink / NVSwitch；
- 是否支持集合通信和 GPUDirect RDMA。

2.2 CPU、内存和 PCIe

- vCPU 与 GPU 的配比；
- 内存容量和带宽；
- NUMA 拓扑；
- PCIe Lane；
- 本地 NVMe 容量和 IOPS；
- 网卡数量、带宽和队列。

2.3 网络

- 实例内 GPU 互联;
- 实例间网络带宽;
- 是否支持 EFA、InfiniBand、RoCEv2 或云厂商专用 RDMA;
- 是否支持 GPUDirect RDMA;
- 可用区和地域延迟;
- 是否支持多轨和拓扑感知放置。

2.4 存储

- 本地 NVMe;
- 云盘 / 块存储 IOPS;
- 共享文件存储;
- 对象存储;
- 并行文件系统;
- 模型缓存;
- 快照、备份和恢复。

2.5 计费

- 按量计费;
- 预留实例 / Capacity Block;
- Spot / Preemptible;
- 包年包月;
- 存储和出网费用;
- 停机是否还收 GPU 费用;
- 跨区域和跨云传输费用。

3. 主要公有云 GPU 入口

平台	官方入口	需要重点确认
AWS	https://aws.amazon.com/ec2/instance-types/accelerated-computing/	P/G 系列、EFA、Capacity Reservation、Spot
AWS EFA	https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/efa.html	GPUDirect RDMA、拓扑和多实例通信
Microsoft Azure	https://learn.microsoft.com/en-us/azure/virtual-machines/sizes-gpu	ND/NC 系列、InfiniBand、区域和配额
Azure InfiniBand	https://learn.microsoft.com/en-us/azure/virtual-machines/extensions/hpc-compute-infiniband-linux	多节点训练网络
Google Cloud	https://cloud.google.com/compute/docs/gpus	A3/A4/G2 等、GPU 配额、区域库存
阿里云	https://www.alibabacloud.com/help/en/ecs/user-guide/gpu-accelerated-compute-optimized-instance-families	GPU ECS、eRDMA、弹性供应
腾讯云	https://www.tencentcloud.com/document/product/213/11518	GPU 计算型实例、网络和存储
百度智能云	https://cloud.baidu.com/product/gpu.html	GPU 云服务器、训练平台
火山引擎	https://www.volcengine.com/product/gpu	GPU 实例、弹性算力
华为云	https://www.huaweicloud.com/	GPU / AI 实例、昇腾生态和 ModelArts

具体实例规格和可用区库存必须以购买时官方控制台为准。

4. 公有云千卡集群规划

4.1 先判断是否真的需要千卡

千卡云集群成本高、配额难、故障面大。先问：

- 能否通过更小模型、量化、LoRA 或蒸馏解决？
- 能否拆成多区域、多集群或多个任务？
- 训练能否用 Spot + Checkpoint？
- 推理是否只需要多副本而不是 TP 千卡？
- 是否存在跨云、跨区域的数据迁移成本？

4.2 容量规划

所需实例数 =
峰值 GPU 需求
/ 单实例 GPU 数
/ 业务利用率
+ 故障冗余

除了 GPU，还要同时申请：

- vCPU 配额；
- 内存配额；
- 本地盘 / 云盘配额；
- 公网 / 内网 IP；
- 负载均衡；
- NAT 和带宽；
- 文件存储 / 对象存储；
- 容器镜像仓库；
- K8s Node 配额；
- 区域库存和预留容量。

4.3 拓扑和故障域

- 尽量把高频通信实例放在同一放置组 / 同一低延迟网络区域；
- TP 不跨高延迟区域；
- 为节点故障预留冗余；
- 检查抢占率和 Spot 回收策略；
- 设计 Checkpoint 和快速恢复；
- 监控多实例通信、慢节点和网络拥塞。

4.4 存储与 Checkpoint

- 训练数据与计算分离；
- 模型权重和 Checkpoint 使用共享存储；
- Checkpoint 并发写入要压测；
- 本地 NVMe 只做缓存，不承担唯一副本；
- 对象存储用于归档和长期版本。

5. 成本控制

5.1 成本公式

```
总成本 =  
GPU 实例费  
+ 存储费  
+ 出网费  
+ 负载均衡 / NAT / 公网 IP  
+ 镜像和日志  
+ 许可证和软件  
+ 运维人力
```

5.2 控制手段

- 标签：业务、团队、模型、环境；
- 预算和告警；
- 定时关机和非工作时间缩容；
- Spot + Checkpoint；
- Reserved / Capacity Block 用于稳定负载；
- 镜像预热和模型缓存；
- 自动缩容到最小副本；
- 删除未使用磁盘和快照；
- 把数据放在同区域，减少出网；
- 定期审计闲置 GPU。

6. 公有云安全

- 使用私有子网，GPU 实例不直接暴露公网；
- SSH 通过堡垒机或 SSM / Session Manager；
- IAM 最小权限；
- Secret Manager 管理密钥；
- 安全组只开放必要端口；
- 对象存储默认私有；
- 加密磁盘、对象和传输；
- 审计 API 调用；
- 镜像扫描和 SBOM；
- 按租户和业务打标签，防止数据串用。

7. 实施步骤

1. 选业务 workload 和目标区域；
2. 申请 GPU、vCPU、存储和网络配额；
3. 选实例族，验证 GPU、NVLink、RDMA 和存储；
4. 建立镜像和初始化脚本；
5. 配置模型缓存、对象存储和共享文件系统；
6. 部署 K8s / Slurm 或云托管训练服务；
7. 部署 vLLM / SGLang / 训练框架；
8. 接入监控、日志、Trace 和成本；

9. 做 Spot 抢占、节点故障和网络故障演练；
10. 对比按量、Spot、预留和自建 TCO。

8. 常见坑

容易犯的问题	正确做法
只按 GPU 型号选实例	同时看 CPU、内存、NVLink、RDMA、存储和配额
跨 AZ 做 TP	高频通信限制在同一低延迟域
Spot 不适合无检查点任务	Checkpoint + 重试 + 回退按量
只算实例费	计算存储、出网、NAT、日志和人力成本
区域库存无保障	预留容量、多个区域或多云方案
自动化关机策略缺失	标签、预算、定时任务和空闲检测
云 SDK 渗透业务代码	标准 API / 兼容层，保留迁移能力
安全组开放 0.0.0.0/0	私网、堡垒机和最小开放端口

9. 视频与资料

- [AWS EC2 加速计算实例](#)
- [AWS EFA 文档](#)
- [Azure GPU VM](#)
- [Azure InfiniBand](#)
- [Google Cloud GPU](#)
- [阿里云 GPU 实例](#)
- [腾讯云 GPU 文档](#)
- [百度智能云 GPU](#)
- [火山引擎 GPU](#)
- [AWS GPU/Spot 讲解](#)
- [Google Cloud GPU VM 教程](#)
- [Azure NVIDIA VM 与 LLM 部署](#)
- [GPU Cluster Network Design](#)
- [How to Reduce Cloud GPU Costs](#)

10. 最终检查表

- ☐ 已确认 GPU 型号、显存、NVLink、RDMA 和区域库存。
- ☐ 已申请 GPU、vCPU、内存、存储和网络配额。
- ☐ 已测试实例间通信和共享存储。
- ☐ 已配置私有网络、IAM、Secret 和安全组。
- ☐ 已有 Checkpoint、重试和 Spot 回退策略。
- ☐ 已有预算、标签、定时关机和空闲资源回收。
- ☐ 已对比按量、Spot、预留和自建成本。
- ☐ 已做跨 AZ、节点故障和网络故障演练。
- ☐ 已保留迁移到其他云或自建集群的路径。

第十部分：32 周完整学习路线（含物理集群扩展说明）

来源文件：05_32周完整学习路线.md

AI Harness / PDE / Infra | 32 周完整学习路线

节奏：每周 10 ~ 12 小时。全职学习可压缩为 16 周，但每个实验的验收物不能省。

主线原则：先业务闭环，再 Harness，再部署，最后平台化。不要先花三个月学 GPU Kernel，也不要只做 UI Demo。

视频版：逐周视频与操作步骤见 09_36周视频教程与逐周操作手册.md。

物理集群增强版：第 1 ~ 32 周完成后，再增加 4 周 GPU 物理集群专项（见 07_GPU物理集群建设选型部署与调优.md 和 08_GPU集群监控调试故障处理与验收.md），总周期为 36 周。

企业落地扩展：避坑见 10，国产 GPU / 千卡见 11，公有云 GPU 见 12。

1. 能力目标

完成本路线后，应该能够：

- 独立定义并交付一个 AI 产品功能；
- 构建带工具、记忆、状态、审批、沙箱、评估和 Trace 的 Harness；
- 用 vLLM 部署模型，并完成压测、量化、TP/PP 和容量分析；
- 在 Kubernetes 上部署多副本推理服务、网关、自动扩缩容和监控；
- 理解 Dynamo、llm-d、KServe、KubeRay、Kueue 等企业组件的位置；
- 用 OWASP LLM Top 10、NIST AI RMF 和 OpenTelemetry 做安全与治理；
- 完成一个可演示、可压测、可复盘、可答辩的毕业项目。

2. 每周固定节奏

时段	内容	产出
周一 1.5h	精读官方文档	概念图和问题清单
周三 2h	写代码 / 配置	可运行 Commit
周五 2h	实验和调试	测试结果、错误记录
周末 3h	压测、评测、报告	Benchmark、评测报告、复盘
月末 2h	阶段评审	演示、风险清单、下月计划

3. 前置要求

- Python：函数、类、异步、类型标注、虚拟环境、测试；
- Linux：文件、进程、权限、网络、Shell；
- Git：分支、Commit、PR、回滚；
- HTTP / JSON / REST / SSE；
- Docker 基础；
- 能读懂 YAML；
- 有基础 SQL 和数据库概念。

弱项补齐规则：

弱项	最短补法
Python 不熟	2 周基础 + 一个小型 FastAPI 项目
Linux / Docker 不熟	1 周命令 + Dockerfile + Compose

弱项	最短补法
K8s 不熟	2 周 Pod / Deployment / Service / ConfigMap / Ingress
LLM 基础不足	1 周 API、Token、Prompt、RAG 概念
前端不熟	2 周 React/Next.js 表单、状态、SSE

4. 32 周路线

第 1 阶段：工程与 AI 基础（第 1~4 周）

周	学习	实验	验收
1	Python、Git、HTTP、JSON、SSE	写一个调用模型 API 的 CLI，支持流式输出	能处理超时、重试、错误码
2	Token、Prompt、Context、模型选择	Prompt 对照实验和结构化输出	有 Prompt 版本和评测表
3	FastAPI、Pydantic、Postgres、Redis	构建最小 API + 任务表 + 状态表	API 有 Schema、错误码和测试
4	Docker、日志、配置、Secrets	把 API Docker 化并本地 Compose 启动	镜像可复现，配置不写死

阶段 1 交付物：foundation/、API 服务、Docker Compose、周报。

第 2 阶段：Harness 基础（第 5~8 周）

周	学习	实验	验收
5	Harness 概念、Agent Loop	实现 Model → Tool → Model 最小循环	有最大步骤、时间、预算
6	Context 与 Memory	短期消息、摘要、长期事实、Artifact	记忆隔离和删除可用
7	Tool Calling / MCP	创建只读 MCP Server 并接入	工具 Schema、超时、审计
8	Run State 与恢复	Run 持久化、取消、恢复、重放	跨进程重启可恢复

阶段 2 交付物：agent-harness/ v0.1、Trace、测试、架构图。

第 3 阶段：Harness 生产化（第 9~12 周）

周	学习	实验	验收
9	Sandbox 与权限	容器沙箱、文件隔离、网络白名单	Prompt 不能绕过权限
10	Human-in-the-Loop	审批服务、状态机、审计	写操作必须审批
11	Verification / Critic	Schema、测试、引用、修复循环	失败不能标成功
12	Long-running Agent	Initializer + Progress + 增量恢复	5 次 Session 重置后继续

阶段 3 交付物：Harness v0.2、审批矩阵、长任务恢复报告。

第 4 阶段：AI PDE 产品工程（第 13~16 周）

周	学习	实验	验收
13	用户研究、JTBD、指标	访谈 3 位用户并建立基线	有真实问题和指标
14	AI PRD、状态机、API 契约	写 Spec 和异常状态	PRD 可被开发执行
15	Next.js / React / SSE / 可访问性	做流式交互原型	包含等待、失败、重试、审批
16	评测、灰度、监控、复盘	接入 Harness API 并灰度	有 Golden Set 和上线计划

阶段 4 交付物：PDE 产品原型、PRD、状态机、评测集、灰度方案。

第 5 阶段：AI Infra 单机与 Kubernetes（第 17~20 周）

周	学习	实验	验收
17	GPU、CUDA、显存、量化	vLLM 单机部署和 API	流式、OOM、显存实验完成
18	KV Cache、PagedAttention、压测	TTFT、Token/s、QPS 压测	有完整 Benchmark 报告
19	Docker、K8s、GPU Operator	部署 vLLM 到 K8s	GPU 调度、探针、Service 正常
20	网关、副本、自动扩缩容	多副本 + LiteLLM / Gateway	滚动升级和回滚可用

阶段 5 交付物：K8s 推理服务、Helm/YAML、压测报告、Runbook。

第 6 阶段：分布式推理与调度（第 21~24 周）

周	学习	实验	验收
21	TP / PP / DP、NCCL	2 卡 TP=2	有加速比和通信数据
22	多机 TP+PP、网络	多节点或模拟多节点部署	能定位网络/显存瓶颈
23	Kueue、配额、优先级	两团队共享 GPU 队列	配额和抢占可解释
24	KV-aware Routing / Dynamo / Ilm-d	部署一个 Well-Lit Path	能解释路由和缓存收益

阶段 6 交付物：多卡基准、调度实验、推理平台选型矩阵。

第 7 阶段：观测、评估、安全与成本（第 25~28 周）

周	学习	实验	验收
25	OTel GenAI、Trace	接入 Trace 和 GPU Metrics	可从投诉追到版本和节点
26	Langfuse / MLflow / promptfoo	建立离线评测和回归门禁	模型/Prompt 变更必须过门禁
27	OWASP LLM Top 10、NIST AI RMF	安全评审、红队和权限演练	有威胁模型和修复记录
28	FinOps、容量、SLO	成本看板、预算告警、混沌演练	有单 Token 成本和降级方案

阶段 7 交付物：统一 Dashboard、评测报告、安全报告、成本和容量报告。

第 8 阶段：毕业项目（第 29~32 周）

周	任务	验收
29	选题、架构、数据、评测基线	有范围和成功标准
30	Harness + 模型 + 工具 + UI 集成	端到端能跑通
31	部署、压测、安全、故障演练	有指标、告警、回滚
32	答辩、README、演示和复盘	通过毕业评分表

毕业项目三选一：

- 企业知识助手：RAG + 引用 + 权限 + Agent + 自部署模型；
- 运维 Copilot：日志 / 指标 / 工单 / 审批 + Harness；
- 数据分析 Agent：Text-to-SQL + 可视化 + 数据权限 + 评测。

5. 每周实验记录模板

Week <N> Lab Report
Goal
Environment
Commands
Result
Metrics
Failure cases
Root cause
Changes made
Reproduction steps
Next step

6. 阶段通关规则

- 没有可复现命令，不算完成；
- 没有指标，不算优化；
- 没有失败案例，不算测试；
- 没有权限和审计，不算生产化；
- 没有评测集，不算 AI 产品；
- 没有成本和回滚，不算企业级。

7. 最终答辩的 12 个问题

1. 用户问题是什么，基线是多少？
2. 为什么不使用普通 API 或规则系统？
3. Harness 的哪些组件是自研，哪些是采购？
4. 工具权限和写操作如何控制？
5. Context、Memory 和 Artifact 如何分层？
6. 模型后端如何替换？
7. vLLM 的瓶颈在哪里，TTFT 和吞吐如何？
8. TP / PP / KV Cache 在平台里如何使用？
9. 评测集、回归门禁和线上指标是什么？
10. Prompt Injection、越权和数据泄露如何防？
11. 成本、容量、SLO 和降级如何管理？
12. 如果流量增长 10 倍，最先优化哪一层？

第十一部分：毕业项目 SOP 与验收

来源文件：06_毕业项目SOP与验收.md

毕业项目 SOP 与验收

1. 项目目标

完成一个可以在企业内部做 PoC 演示的 AI 产品，不做“只会聊天的 Demo”。

推荐选题：

选题	领域	核心能力
企业知识助手	知识管理 / 客服 / 员工支持	RAG、权限、引用、Agent、自部署模型
运维 Copilot	SRE / IT 运维	指标、日志、工单、工具调用、审批、恢复
数据分析 Agent	经营分析 / 数据平台	Text-to-SQL、权限、图表、校验、审计

2. 毕业项目必须覆盖

PDE 层：用户问题、PRD、交互、状态、反馈、上线
Harness 层：模型、上下文、记忆、工具、审批、沙箱、恢复、评测
Infra 层：模型服务、网关、多副本、监控、安全、成本
物理集群层（选 GPU 专项时）：机柜、供电、制冷、网络、存储、Burn-in、监控、故障处理

3. 推荐目录

graduation-project/ ├── README.md ├── docs/ │ ├── architecture.md │ ├── product-spec.md │ ├── threat-model.md │ ├── runbook.md │ └── benchmark.md ├── apps/ │ ├── web/ │ ├── api/ │ └── worker/ ├── harness/ │ ├── controller.py │ ├── context.py │ ├── memory.py │ ├── policy.py │ ├── sandbox.py │ └── verifier.py ├── tools/ │ ├── registry.py │ └── mcp_servers/ ├── evals/ │ ├── datasets/ │ └── suites/ ├── deploy/ │ ├── docker-compose.yml │ ├── helm/ │ └── k8s/ ├── benchmarks/ ├── reports/ └── scripts/
--

4. 端到端验收流程

第一步：启动

```
cp .env.example .env
docker compose up -d
curl http://localhost:8080/health
```

验收：一条命令启动，README 能让另一个工程师在 30 分钟内复现。

第二步：产品流程

打开前端，完成：

1. 用户提出问题；
2. 系统显示“正在检索 / 调用工具”；
3. 返回带引用或证据的答案；
4. 遇到高风险操作时请求审批；
5. 工具失败时显示可恢复错误；
6. 用户可以取消、重试、反馈；
7. 人工接管后保留完整上下文。

第三步：Harness 流程

验证：

- 最大步骤、时间、Token、成本上限；
- 工具 Schema、权限、幂等和审计；
- Run 停止后可恢复；
- 模型替换后业务 API 不变；
- 所有步骤有 Trace。

第四步：Infra 流程

验证：

- vLLM 提供 OpenAI 兼容接口；
- 通过模型网关访问，不直连实例；
- 至少 2 个副本；
- 滚动升级和回滚；
- GPU、延迟、吞吐、错误、成本有看板；
- 故障副本不会导致整个服务不可用；
- 有配额和限流。

第五步：评测

至少建立：

```
golden_set    30 条真实任务
ambiguous_set 20 条模糊问题
failure_set   20 条工具/权限/超时失败
adversarial_set 20 条注入/越权/数据外泄
```

门禁：

- 任务成功率不得低于上一版；
- 安全用例不得出现高危通过；

- P95 延迟不得超过 SLO；
- 成本不得超过预算；
- 人工接管率不能明显恶化。

第六步：压测

并发梯度：

1, 4, 8, 16, 32, 64

记录：

指标	1	4	8	16	32	64
TTFT P95						
TPOT P95						
Output token/s						
QPS						
错误率						
GPU 利用率						
KV Cache						
每百万 Token 成本						

第七步：故障演练

至少做 5 个：

1. 杀掉一个模型副本；
2. 模拟模型加载失败；
3. 工具超时；
4. Redis / 数据库短暂不可用；
5. 达到租户配额或 Token 预算。

每个演练记录：

- 现象；
- 影响范围；
- 自动恢复时间；
- 人工处理步骤；
- 预防措施。

第八步：安全演练

- Prompt Injection：文档中植入恶意指令；
- 越权工具：请求未授权工具；
- 数据泄露：诱导输出其他租户信息；
- 供应链：模拟过期 / 漏洞镜像；
- 日志泄露：检查 Trace 和日志中的 PII；
- 无界消费：制造无限循环和 Token 消耗。

5. 必须提交的文件

文件	内容
README.md	启动、架构、目录、演示步骤
product-spec.md	用户、问题、状态机、SLO、评测
architecture.md	Harness + Infra 架构和数据流
threat-model.md	资产、威胁、控制、残余风险
benchmark.md	压测配置、指标、图表、结论
runbook.md	告警、降级、回滚、人工接管
scorecard.md	自评分与证据链接

6. 评分表

维度	权重	及格	优秀
用户问题与产品闭环	15	有真实问题和流程	有基线、灰度和复盘数据
Harness 控制面	20	有工具和状态	有记忆、审批、沙箱、恢复、验证
模型部署与推理	15	vLLM 可运行	有压测、量化、缓存和容量分析
K8s 与平台化	15	有 Deployment	多副本、网关、调度、扩缩容、回滚
评测与可观测性	15	有 Trace 和评测集	回归门禁、线上指标、错误归因
安全与合规	10	有 RBAC 和 Secrets	威胁模型、红队、DLP、审计
工程与可复现	10	README 可启动	一键启动、CI、版本锁定、完整证据

总分：

- < 60: Demo，不能算企业落地；
- 60 ~ 74: 可做内部试验；
- 75 ~ 89: 可做受控试点；
- 90+: 具备进入企业平台评审的基础。

7. 答辩四分钟结构

0:00 - 0:30 用户问题、基线和目标
0:30 - 1:30 Harness 架构和关键控制
1:30 - 2:30 Infra 部署、压测和容量
2:30 - 3:15 评测、安全、失败恢复
3:15 - 4:00 数据结果、不足和下一步

答辩中必须展示：

- 一次正常任务；
- 一次需要审批的任务；
- 一次工具失败并恢复；
- 一次模型副本故障；
- 一次评测回归对比；
- 一张成本与延迟看板。

第十二部分：最新资料与来源索引

来源文件：01_最新资料与来源索引.md

最新资料与来源索引

检索与核验日期：**2026-10-04**。优先保留官方文档、协议规范、核心开源项目和能说明生产实践的行业资料。第三方文章只用于补充观点，不作为安全和接口唯一依据。

一、AI Harness / Agent Runtime

资料	类型	重点	用途
Microsoft Agent Framework: Agent Harness	官方文档	把 Harness 定义为模型周围的运行时脚手架，驱动模型和工具调用、会话状态、审批策略、长任务推进	作为 Harness 组件清单的主参考
BCG: Harness Engineering: The Operating System for Agentic AI	行业研究，2026-09-16	Harness 是企业 Agent 的操作系统；强调 start small、build don't buy、选择性落地	企业组织、采购和治理视角
Parallel: What is an agent harness?	深度解读，2026-09-26	Intent、Orchestration、Tool Execution、Context、Memory、Verification、Persistence	理解 Harness 与 Prompt Engineering / Framework 的边界
Anthropic: Building effective agents	工程博客，2024-12-19	Workflow vs Agent、Prompt Chaining、Routing、Parallelization、Orchestrator、Evaluator	建立最简单的 Agent 设计原则；注意文中旧框架已标注变化
Anthropic: Effective harnesses for long-running agents	工程博客，2025-11-26	Initializer Agent、Progress Log、跨 Context Window 恢复、增量交付	长任务 Harness 的关键设计
Anthropic: Writing effective tools for agents	工程博客，2025-09-11	工具边界、命名空间、返回上下文、Token 效率、工具评测	设计企业级 Tool / MCP
OpenAI: Custom instructions with AGENTS.md	官方文档	全局、仓库、子目录 AGENTS.md 的优先级与覆盖机制	为 Codex 类编码 Harness 建立团队规则
OpenAI: Codex Skills	官方文档	SKILL.md、渐进式披露、仓库/用户/管理员范围、显式与隐式触发	构建可复用工作流和领域能力
OpenAI: Agents API Guide	官方文档	Responses、状态、工具、后台运行、多 Agent、评测和生产建议	对比托管 Agent 能力与自研 Harness
OpenAI: Codex Administration	官方文档	企业部署、权限、策略和治理入口	大企业推广 AI Coding / Agent 的治理参考
Model Context Protocol	开放协议	Host / Client / Server、工具、资源和提示词协议；当前文档版本含 2026-07-28	工具接入的标准协议
A2A Protocol	开放协议	Agent-to-Agent 通信、发现、任务与协作	多 Agent / 多团队互操作
Open Agent Skills	开放标准	用 SKILL.md 打包可复用 Agent 能力	让团队能力可版本化、可分发
Nous Research Hermes Agent	开源参考	自我改进、长期记忆、Harness 生态	观察个人 Agent Harness 设计，不建议直接照搬到企业
OpenClaw	开源参考	本机运行、插件、记忆、多渠道 Surface	观察个人助理型 Harness，重点研究隔离和权限
ByteDance DeerFlow	开源参考	长任务 SuperAgent Harness、Sandbox、Memory、Tools、Skills、Subagents、Message Gateway	研究生产级长任务 Harness 的架构拆分

二、AI PDE：产品设计 / 产品开发工程

资料	类型	重点	用途
FDE Academy: AI Product Engineer	岗位解析, 2026-08-26	AI Product Engineer 端到端负责问题、原型、评估、上线和迭代	PDE 能力模型的直接参考
PDE 新职业解析	中文行业文章, 2026-08-11	PDE = Product Design Engineer; 设计、交互、生产代码三合一	中文岗位语境和转型路径
Nielsen Norman Group: AI Chat UX	UX 研究	Chat 界面的用户预期、可见性和信任问题	设计 AI 交互状态
Google PAIR: People + AI Guidebook	产品设计指南	用户价值、反馈、错误恢复、信任和解释	AI 产品设计基础
Microsoft HAX Toolkit	人机交互指南	不确定性、错误、可解释性、用户控制	补足 AI 产品的交互异常状态

PDE 的工程部分继续使用：

- [MDN Web Docs](#): HTML / CSS / JavaScript / Accessibility
- [React](#): 组件、状态和交互
- [Next.js](#): 全栈产品页面与 API
- [FastAPI](#): 后端 API
- [Pydantic](#): 结构化输入输出与校验

三、AI Infra：推理、编排、调度和平台

3.1 推理引擎

工具	官方入口	适用位置
vLLM	docs.vllm.ai	主推理引擎, PagedAttention、Continuous Batching、OpenAI API
SGLang	docs.sglang.ai	RadixAttention、复杂控制流、Agent / RAG 负载
TensorRT-LLM	NVIDIA TensorRT-LLM Docs	NVIDIA 深度优化和编译优化
NVIDIA NIM	NVIDIA NIM LLM Docs	企业模型容器和标准化推理服务
NVIDIA Dynamo	NVIDIA Dynamo Docs	分布式推理、Disaggregated Serving、KV-aware Routing、缓存、自动扩缩容

3.2 Kubernetes 服务与路由

工具	官方入口	作用
Ray Serve LLM	Ray Serve LLM	Python 原生 LLM 服务、副本、扩缩容、组合式应用
KubeRay	Ray on Kubernetes	在 K8s 上运行 Ray 集群和 Ray Serve
KServe	KServe	生成式与预测式推理平台、Canary、模型缓存、自动扩缩容
Gateway API Inference Extension	项目文档	Inference Gateway、Endpoint Picker、前缀缓存和推理路由
llm-d	llm-d.ai	Kubernetes 原生分布式 LLM 推理, Well-Lit Paths, 多硬件
vLLM Production Stack	GitHub	vLLM 的 K8s 参考栈、路由、KV Cache Offloading、监控
LiteLLM	docs.litellm.ai	多模型网关、配额、路由和 OpenAI 兼容接口

3.3 调度、训练与模型生命周期

工具	官方入口	作用
Kueue	kueue.sigs.k8s.io	队列、配额、优先级、抢占、多租户 GPU 共享
Kubeflow Trainer	Kubeflow Trainer	K8s 原生分布式训练和微调
GPU Operator	NVIDIA GPU Operator	GPU 驱动、设备插件、监控和节点管理
MLflow GenAI	MLflow for Agents and LLMs	Tracing、评估、Prompt、模型和 Agent 生命周期

3.4 观测、评估与安全

工具 / 标准	官方入口	作用
OpenTelemetry GenAI	Semantic Conventions	LLM / Agent 的 Trace、Span、Token、工具调用标准
Langfuse	Langfuse Docs	LLM / Agent Tracing、Prompt、评估、成本监控
Prometheus	prometheus.io	基础设施和 GPU 指标
Grafana	grafana.com/docs	服务、GPU、成本和业务指标可视化
promptfoo	promptfoo Docs	Prompt / Agent 红队与回归评测
Ragas	Ragas Docs	RAG 检索和生成质量评估
DeepEval	GitHub	LLM / Agent 单元测试式评估
OWASP LLM Top 10	GenAI Security Project	Prompt Injection、敏感信息、供应链、过度代理等风险
NIST AI RMF	AI Risk Management Framework	企业 AI 风险和治理框架
NVIDIA NeMo Guardrails	官方文档	输入、输出、工具和对话安全护栏

四、GPU 物理集群：选型、建设、调优、监控和故障

资料	类型	重点
NVIDIA DGX SuperPOD 文档入口	官方架构与部署文档	AI Factory / SuperPOD 的 GPU、网络、存储、电力和部署参考
NVIDIA DGX B300	官方产品	当前 DGX 平台和集群级部署能力
NVIDIA GB300 NVL72	官方产品	机架级 GPU 互联、液冷和高密度 AI Factory 参考
NVIDIA GB200 NVL72	官方产品	机架级 NVLink 域和分布式训练 / 推理参考
NVIDIA GPU Operator	官方文档	K8s 驱动、Container Toolkit、Device Plugin、DCGM、MIG 和 RDMA
GPU Operator RDMA	官方文档	RDMA 与 GPUDirect RDMA 在 K8s 上的配置
GPU Operator MIG	官方文档	MIG 实例、隔离和资源配置
NVIDIA Network Operator	开源项目	K8s 上的 Mellanox / NVIDIA NIC、RDMA、SR-IOV 和网络配置
NVIDIA DCGM	官方文档	GPU 遥测、健康诊断、策略和指标
DCGM Diagnostics	官方文档	GPU 诊断等级、健康检查和 Burn-in
NVIDIA NCCL	官方文档	多卡 / 多机集合通信和调试
NVIDIA XID Errors	官方文档	GPU 驱动 / 硬件错误码解释
NVIDIA GPU Debug Guidelines	官方文档	GPU 故障排查和日志定位
NVIDIA nvbandwidth	官方工具	GPU / NVLink / PCIe / P2P 带宽测试
NVIDIA nccl-tests	官方工具	All-Reduce、All-Gather 等集合通信基准
gpu-burn	开源工具	GPU 稳定性、功耗、温度和 ECC Burn-in
Kubernetes Dynamic Resource Allocation	官方文档	新一代 GPU 等设备资源分配
Kueue	官方文档	队列、配额、优先级、抢占和 AI workload 调度
Slurm Documentation	官方文档	HPC / 训练集群的作业调度、分区和 QoS
Node Problem Detector	开源项目	节点硬件和内核问题检测
OpenBMC	开放项目	带外管理、BMC 固件和服务器生命周期
MAAS	官方文档	裸机生命周期、PXE、自动化和资产管理
Tinkerbell	开源项目	裸机 provisioning 和工作流
Redfish / DMTF	标准	服务器带外管理和自动化接口

五、国产 GPU、千卡集群与公有云

国产算力

平台	官方入口	重点
华为昇腾	hiascend.com	Atlas、CANN、MindSpore、MindIE、MindCluster、HCCL
昇腾 CANN	CANN	驱动、算子、编译器、运行时和迁移
MindIE	MindIE	大模型推理服务
MindCluster	MindCluster	集群训练、推理和运维
华为超节点	SuperPoD	大规模互联和超节点架构
寒武纪	cambricon.com	思元 / MLU 及软件生态
海光	hygon.cn	DCU / 深算生态
摩尔线程	mthreads.com / 开发者平台	MUSA、KUAE、MUSA Deploy、云原生
壁仞科技	birentech.com	壁砺 GPU、BIRENSUPA、光互连
昆仑芯	kunlunxin.com	R200 / R480、XPU 和集群
沐曦	metax-tech.com	MXMACA、GPU 超节点
天数智芯	iluvatar.com	天垓、彤央、智铠和软件栈

千卡集群

资料	重点
NVIDIA DGX SuperPOD	超大规模 GPU、网络、存储、电力和部署参考
Kueue	多租户队列、配额、抢占和多集群调度
Slurm	HPC / 训练作业调度
NVIDIA NCCL	集合通信、拓扑和调试

公有云 GPU

平台	官方入口	重点
AWS	加速计算实例	GPU 实例、EFA、Capacity Block、Spot
AWS EFA	EFA Docs	RDMA / GPUDirect RDMA
Azure	GPU VM Sizes	ND/NC 系列和 InfiniBand
Azure InfiniBand	文档	多节点训练互连
Google Cloud	GPU Docs	GPU 配额、区域库存和实例族
阿里云	GPU 实例	ECS GPU、网络和弹性
腾讯云	GPU 文档	GPU 实例和存储
百度智能云	GPU	GPU 云服务器和训练平台
火山引擎	GPU	弹性 GPU 实例
华为云	huaweicloud.com	GPU / AI 实例和昇腾生态

六、资料可信度规则

学习时按以下优先级选资料：

1. **官方文档 / 协议规范**：接口、参数、安全和兼容性以官方为准。
2. **官方工程博客**：理解设计动机和生产经验。
3. **核心开源仓库**：看当前 issue、release、架构和实际代码。
4. **行业研究 / 咨询文章**：用于组织、治理和落地方法，不替代技术验证。

5. 个人教程 / 视频：只用于入门和建立直觉，必须回到官方文档验证。

七、高频版本风险

- vLLM、SGLang、Dynamo、llm-d、Ray、KServe 的接口和部署方式变化较快，禁止在简历或生产中写死一个旧命令。
- MCP、A2A、Agent Skills 仍可能演进，接入时保留适配层，不要让协议渗入业务核心。
- GPU Operator、CUDA、驱动、推理引擎和模型量化格式存在兼容矩阵，部署前必须查对应版本。
- Harness 的 Tool 权限、数据出口和日志中可能包含敏感信息；安全评审必须先于大规模推广。
- 不要把第三方的未验证 Benchmark 当作采购结论，必须用自己的 workload 和数据复测。
- GPU 代际、功耗、液冷、交换机、光模块和固件必须按整机 / 整柜验证，不能只看单卡参数。
- RoCE / InfiniBand 的 PFC、ECN、QoS、GID、MTU 和 rail 拓扑错误会表现为 NCCL 超时或吞吐异常。
- BMC / Redfish 网络必须与业务网隔离，默认凭据、固件漏洞和横向移动风险要纳入安全评审。